

Deep Defense Design: The Thunder Hash Family

Meiqin Wang^{1,3,5}, Kai Hu^{1,3,4}, Hao Lei^{1,3}, Yanhong Fan^{1,3}, Lixuan Wu^{1,3},
Yong Fu^{1,3,5}, Shiyao Chen^{1,3}, Jiamin Cui^{1,3}, Zhongfeng Niu², Qun Liu^{1,3},
Chenhao Jia^{1,3} and Longzheng Cui^{1,3}

¹ School of Cyber Science and Technology, Shandong University, Qingdao, China

mqwang@sdu.edu.cn, kai.hu@sdu.edu.cn, leihao@mail.sdu.edu.cn,

² School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore

³ State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China

⁴ Suzhou Research Institute, Shandong University, Suzhou, 215123, China

⁵ Quan Cheng Shandong Laboratory, Jinan, China

1 Introduction

Cryptographic hash functions are a core building block of modern cryptography. They underlie digital signatures, message authentication, key derivation, and commitments, and their strength is measured chiefly by resistance to collision, preimage, and second-preimage attacks. The ongoing migration to post-quantum cryptography raises their importance further. Hash functions are the sole primitive behind hash-based signatures [Mer79, LM95, CAD⁺20], one of the most conservative families of post-quantum signatures. They also remain robust against quantum adversaries on their own: Grover’s algorithm [Gro96] speeds up preimage search only quadratically, and quantum collision search [BHT98] yields a limited advantage, so an h -bit classical security level retains a substantial margin in the quantum setting. High security levels, such as 1024-bit preimage resistance, therefore grow more attractive as quantum computing advances. Driven by these demands, the Chinese call for next-generation commercial cryptographic algorithms [NIC25] seeks the successors of the current SM3 standard [Sta10], targeting security levels up to 1024 bits with strong (second) preimage resistance in addition to collision resistance. SM3 provides a 256-bit digest, with 128-bit collision and 256-bit (second) preimage resistance, while Thunder is designed for the higher levels the call targets, reaching up to 512-bit collision and 1024-bit (second) preimage resistance.

Many recent hash designs are permutation-based and follow the sponge construction [BDPVA07, GJMG11] that the SHA-3 standard [D⁺15] made popular. A sponge with capacity c provides only about $c/2$ bits of (second) preimage resistance [AMP11, LM22]. Reaching h -bit (second) preimage security therefore forces $c \geq 2h$, and SHA-3 follows this rule with $c = 2h$. A 1024-bit (second) preimage target then forces $c = 2048$ and a permutation of width above 2048 bits. Such a permutation is expensive to design and to evaluate, and its cost is paid by every member of the family.

To overcome this, we build on a new mode of operation. The sponge-with-feed-forward XOF construction $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ of Guo et al. [GHJ⁺25], to appear at CRYPTO 2026, reaches high (second) preimage security with a substantially smaller permutation than a standard sponge requires. It feeds the inner part of the state forward at each permutation call. This raises (second) preimage resistance close to the output length h while keeping the capacity at $c \approx h+64$, instead of $c \geq 2h$. A constant injected in the final block prevents the length-extension attacks that squeezing the inner part would otherwise allow [AMP12,

Table 1: Overview of the members in the Thunder family. For the XOF instances, ν denotes the arbitrary output length, and the preimage and collision security are capped at the target level by the capacity.

Name	Size		Security		Data limits (bits)
	State	Hash	Preimage	Collision	
Thunder-1024	1,600	1024	1024	512	2^{64}
Thunder-768	1,600	768	768	384	2^{64}
Thunder-512	1,600	512	512	256	2^{64}
Thunder-XOF-256	1,600	ν	$\min\{256, \nu\}$	$\min\{256, \nu/2\}$	2^{64}
Thunder-XOF-384	1,600	ν	$\min\{384, \nu\}$	$\min\{384, \nu/2\}$	2^{64}
Thunder-XOF-512	1,600	ν	$\min\{512, \nu\}$	$\min\{512, \nu/2\}$	2^{64}

HPY12]. Its collision, preimage, and indistinguishability security are established in the random-permutation model.

We instantiate $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ into Thunder, a family of fixed-length hash functions and arbitrary-length extendable-output functions (XOFs) built on a single 1600-bit permutation, *Thunder- p* . A 1600-bit width then reaches every (second) preimage target up to 1024 bits, where a standard sponge would need a permutation of width above 2048 bits. The members of the Thunder hash family are

- Thunder-1024
- Thunder-768
- Thunder-512
- Thunder-XOF-256
- Thunder-XOF-384
- Thunder-XOF-512

differentiated by their claimed security level against preimage attacks. The details of the six members are shown in Table 1. For example, *Thunder-1024* provides 1024-bit security against a preimage attack.

The state transformation shared by all members is a new 1600-bit permutation, *Thunder- p* . It is organized as a 5×5 array of 64-bit lanes, the lane-based geometry that KECCAK- $f[1600]$ [D⁺15] made standard. Each round applies five operations,

$$R = \psi \circ \pi \circ \theta \circ \rho \circ \iota,$$

namely a sparse round constant (ι), a lane-wise cyclic shift (ρ), a linear diffusion layer (θ), a lane permutation (π), and a 5-bit nonlinear substitution (ψ). *Thunder- p* is designed so that each round does more cryptographic work, which lets the permutation reach its security target in fewer rounds. Two choices drive this. The diffusion layer θ is a symmetric circulant twin column-parity mixer [LRL⁺24] with differential and linear branch number 12, three times that of the conventional column-parity mixer used in KECCAK-like designs, so low-weight differential and linear patterns cannot survive a single mixing step. The nonlinear layer ψ is a 5-bit S-box of algebraic degree 3, applied along the y -axis, with differential uniformity 14 and linearity 24. A stronger mixing layer and a cubic S-box make trails decay and the algebraic degree grow faster per round, so 12 rounds suffice where

KECCAK- $f[1600]$ uses 24. This reduction is the source of **Thunder**'s efficiency advantage over SHA-3, and it still leaves a deep margin: our trail searches and degree analysis place the best distinguishers at roughly half the round count.

Thunder is efficient in software on 64-bit platforms and competitive in hardware. On x86-64 (Intel Xeon Platinum 8358) and ARM (Rockchip RK3588), the optimized software reaches 6.95 and 7.62 cycles per byte for **Thunder**-512, 9.28 and 10.14 for **Thunder**-768, and 13.81 and 15.27 for **Thunder**-1024 on long messages. In a 55nm ASIC synthesis, every fixed-output instance exceeds the throughput of SHA3-512; for long messages **Thunder**-512 raises throughput by 82% and the throughput-to-area ratio by 16%, since its permutation runs in half the cycles of KECCAK- $f[1600]$. Against SM3, **Thunder**-512 is competitive in software, faster at every measured length on x86-64 and at short and long messages on ARM, while **Thunder**-768 and **Thunder**-1024 trade a moderate software cost for their higher security levels.

Contributions. The contributions of this work are as follows.

- We design the **Thunder** hash family. It instantiates the sponge-forward mode $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ [GHJ+25] over a single 1600-bit permutation, and it reaches up to 1024-bit (second) preimage and 512-bit collision security without the $c \geq 2h$ capacity that a standard sponge requires.
- We design **Thunder**- p , a new 1600-bit permutation in the KECCAK lane geometry. Its round function pairs a symmetric circulant twin column-parity mixer of branch number 12 with a degree-3 5-bit S-box, iterated for 12 rounds.
- We analyze the security of **Thunder**. The analysis covers the mode (collision, preimage, second-preimage, and indistinguishability bounds in the random-permutation model, together with quantum bounds), the permutation (differential, linear, impossible-differential, zero-correlation, higher-order differential, integral, and differential-linear properties), and the hash (collision under differential and rebound attacks, internal-differential attacks, preimage via differential-linear and linear-structure distinguishers, meet-in-the-middle preimage attacks, statistical randomness tests, and length-extension resistance).
- We implement **Thunder** in software and hardware. The optimized software runs on x86-64 and ARM, and a 55nm ASIC synthesis of the three fixed-output instances achieves higher throughput than SHA3-512 at every tested message length.

The remainder of this document is organized as follows. We first specify the **Thunder** family and the permutation **Thunder**- p . We then present the design rationale, including the parameter choices that meet the security targets. The security analysis follows, treating the mode, the permutation, and the hash function in turn. We close with the software and hardware performance results and a discussion.

2 Specification

2.1 Notations

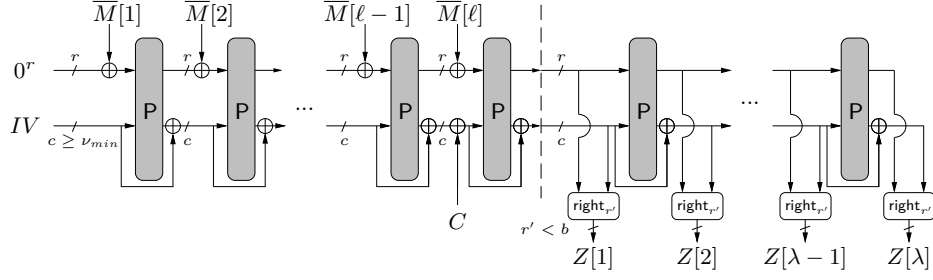


Figure 1: Sponge-with-feed-forward XOF construction $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$, when outputting ν bits ($t = \nu \bmod c$).

r	the bit size of the outer part, a.k.a. rate
c	the bit size of the inner part, a.k.a. capacity
b	the bit size of the internal state, i.e., the size of the permutation
IV	initial vector
$\text{right}_c(X)$	the lower c bits of X
$\text{left}_\nu(X)$	the upper (leftmost) ν bits of X
$a b$	the concatenation of a and b
h	fixed digest length in bits, $h < c$ (here $h \in \{512, 768, 1024\}$)
ν	XOF output length in bits, $\nu_{\min} \leq \nu \leq \nu_{\max}$
n_c, n_r, n_d	number of 64-bit capacity / rate / digest lanes, $n_c + n_r = 25$, $n_d = h/64$
r'	the squeezing rate: output bits produced per permutation call (may exceed r)
$\text{pd}10^*$	the injective 10^* padding: $M 10^{r-(M +1 \bmod r)}$

2.2 Sponge-forward mode

$\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ is the sponge-with-feed-forward construction of Guo et al. [GHJ⁺25], defined upon a permutation $P : \{0, 1\}^b \mapsto \{0, 1\}^b$ and the injective padding function

$$\text{pd}10^*(M) := M||10^{r-(|M|+1 \bmod r)}$$

of [GJMG11]. Its minimal and maximal output lengths are $\nu_{\min}$ and $\nu_{\max}$ respectively. Let $M \in \{0, 1\}^*$ and ν be an integer such that $\nu_{\min} \leq \nu \leq \nu_{\max}$. On input (M, ν) , $\text{Sponge-F}^{\text{P}, \text{pd}10^*}(M, \nu)$ outputs a string $Z \in \{0, 1\}^\nu$. $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ has two additional parameters $r, c \in \mathbb{N}$ and a fixed public constant $IV \in \{0, 1\}^c$, where $c \geq \nu_{\min}$, $r + c = b$, where r , resp. c , is the bit length of the “sponge” outer, resp. inner, part of the b -bit internal state. The concrete process of $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ is described by the algorithms in Fig. 2 and illustrated in Fig. 1.

$\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ can be naturally instantiated as a XOF. For a fixed digest length $h < c$, it instead yields a fixed-output hash with squeezing rate $r' = h$. A single squeeze then produces the whole digest ($\lambda = \lceil h/r' \rceil = 1$), so the squeezing phase makes no additional permutation call, and the digest is read directly from the inner part of the final state. We depict this case in Fig. 4 and write

$$\text{Sponge-F}_h^{\text{P}, \text{pd}10^*}(\cdot) := \text{Sponge-F}^{\text{P}, \text{pd}10^*}(\cdot, h), \quad (1)$$

for the corresponding fixed-output instantiation.

```

1: Procedure Sponge- $F^P, \text{pd}10^*(M, \nu)$ 
2:  $(\overline{M}[1], \dots, \overline{M}[\ell]) \xleftarrow{r} \text{pd}10^*(M), X \leftarrow 0^r \| IV$ 
3: // Absorbing phase
4: for  $i = 1, 2, \dots, \ell$  do
5:   if  $i \neq \ell$  then
6:      $X \leftarrow X \oplus (\overline{M}[i] \| 0^c), S \leftarrow \text{right}_c(X)$ 
7:   else //  $i = \ell$ 
8:      $X \leftarrow X \oplus (\overline{M}[i] \| C), S \leftarrow \text{right}_c(X)$ 
9:    $X \leftarrow P(X) \oplus (0^r \| S)$ 
10: // Squeezing phase
11:  $\lambda \leftarrow \lceil \frac{\nu}{r'} \rceil$ 
12: for  $i = 1, 2, \dots, \lambda$  do
13:    $Z[i] \leftarrow \text{right}_{r'}(X)$ 
14:    $S \leftarrow \text{right}_c(X)$ 
15:   if  $i < \lambda$  then
16:      $X \leftarrow P(X) \oplus (0^r \| S)$ 
17: return  $\text{left}_\nu(Z[1] \| \dots \| Z[\lambda])$ 

```

Figure 2: Sponge- $F^P, \text{pd}10^*$: the XOF construction built upon the sponge with feed-forward construction, with parameters r, c, r' and the one-zero injective padding $\text{pd}10^*$.

2.3 Specification of Thunder- p

State organization. The internal state of Thunder- p has size 1600 bits and is organized as a $5 \times 5 \times 64$ array, as shown in Fig. 3,

$$D \in \mathbb{F}_2^{5 \times 5 \times 64}.$$

The bit at coordinate (x, y, z) is denoted by $D[x][y][z]$, where $x, y \in \mathbb{Z}_5$ and $z \in \mathbb{Z}_{64}$. Here, x denotes the column coordinate, y denotes the row coordinate, and z denotes the bit position inside a 64-bit lane.

Equivalently, the state may be viewed as a one-dimensional array $S \in \mathbb{F}_2^{1600}$, with the mapping

$$S[64(5y + x) + z] = D[x][y][z].$$

Round function. The permutation of Thunder- p consists of 12 rounds. Each round applies the following sequence of transformations:

$$R = \psi \circ \pi \circ \theta \circ \rho \circ \iota.$$

All additions are over $\mathbb{F}_2$. Unless otherwise stated, the coordinates x and y are taken modulo 5, and the coordinate z is taken modulo 64.

The ι operation. The ι operation adds a round constant to lane $(0, 0)$. For the i -th round, it is defined as

$$D'[0][0] = D[0][0] \oplus RC_i,$$

and

$$D'[x][y] = D[x][y]$$

for all $(x, y) \neq (0, 0)$. The 12 round constants are listed in Table 2.

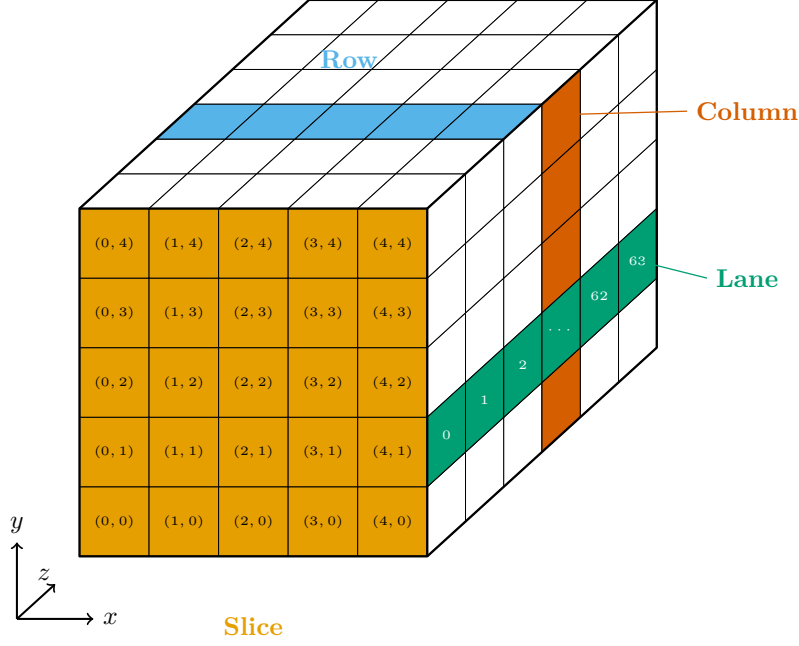


Figure 3: The state organization of Thunder- p .

The ρ operation. The ρ operation cyclically shifts each 64-bit lane along the z -axis:

$$D'[x][y][z] = D[x][y][z + \tau(x, y)].$$

The shift offsets $\tau(x, y)$ are given in Table 3.

The θ operation. The θ operation is a linear diffusion layer based on two column parity planes. First, define

$$P[x][z] = \bigoplus_{y=0}^4 D[x][y][z],$$

and

$$Q[x][z] = \bigoplus_{y=0}^4 D[x][y][z + t_y].$$

Then compute

$$E[x][z] = P[x][z] \oplus P[x-1][z+r],$$

and

$$F[x][z] = Q[x+1][z] \oplus Q[x-2][z+s].$$

Table 2: Round constants of the Thunder- p .

Round i	RC_i	Round i	RC_i
0	0x0000000000000001	6	0x0000000000000009
1	0x0000000000000003	7	0x00000000000000083
2	0x000000000000000A	8	0x000000000000800A
3	0x0000000000000088	9	0x0000000000000089
4	0x0000000000008081	10	0x0000000000008083
5	0x0000000000008002	11	0x000000000000800B

Table 3: Thunder- p ρ operation parameters $\tau(x, y)$.

	$x =$ 0	$x =$ 1	$x =$ 2	$x =$ 3	$x =$ 4
$y = 4$	6	18	49	17	43
$y = 3$	48	30	61	8	38
$y = 2$	34	5	0	2	37
$y = 1$	40	52	19	47	9
$y = 0$	45	33	13	1	27

The output of θ is defined as

$$D'[x][y][z] = D[x][y][z] \oplus E[x][z + u] \oplus F[x][z + u + t_y].$$

The parameters used in θ are listed in Table 4.

Table 4: θ operation parameters.

y	r	s	t_y	u
0	1	36	41	26
1	1	36	50	26
2	1	36	6	26
3	1	36	10	26
4	1	36	27	26

The π operation. The π operation permutes the 25 lanes in the xy -plane. It is defined as

$$D'[y][2x + 3y + 2][z] = D[x][y][z].$$

The ψ operation. The ψ operation is the nonlinear layer of Thunder- p . For each fixed pair (x, z) , it acts on the five bits along the y -axis:

$$D'[x][y][z] = D[x][y + 3][z] \oplus \overline{D[x][y + 1][z]} D[x][y + 2][z] \overline{D[x][y + 4][z]},$$

where $\bar{a} = a \oplus 1$ for $a \in \mathbb{F}_2$.

2.4 Thunder Hash

The Thunder hash function instantiates the sponge-forward mode of Fig. 2 with the 12-round Thunder- p permutation. It provides three fixed-output instances, Thunder-512, Thunder-768, and Thunder-1024, with output lengths 512, 768, and 1024 bits. Each instance evaluates $\text{Sponge-F}_h^{\text{P, pd10}^*}$ of Eq.(1) for its output length h . In these three instances, we let $c = h + 64$ for every instance, the squeezing phase consists of no additional permutation call, and the digest is read directly from the inner part of the final state. Fig. 4 illustrates the three Thunder instances.

2.4.1 Instance Parameters

For each Thunder instance, the state size is $b = 1600$ bits and the underlying primitive is the 12-round Thunder- p permutation specified above. The instance-dependent parameters are the rate r , the capacity c , the output length h , and a public initialization vector. Since the rate and capacity are lane-aligned, we write

$$n_c = c/64, \quad n_r = r/64$$

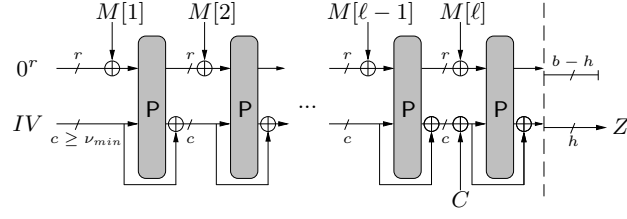


Figure 4: Simplified illustration for the conventional fixed-output-length hash with output length $h < c$.

Table 5: Instance parameters and security claims for **Thunder** hash computation. Collision and preimage security are stated as design targets, in bits.

Instance	Parameters					Security (bits)	
	h	c	r	n_c	n_r	Collision	Preimage
Thunder-512	512	576	1024	9	16	256	512
Thunder-768	768	832	768	13	12	384	768
Thunder-1024	1024	1088	512	17	8	512	1024

for the corresponding numbers of 64-bit capacity and rate lanes, so that $n_c + n_r = 25$. The parameters of the three fixed-output instances are given in Table 5.

Each instance is initialized with a public IV of n_c 64-bit lanes. We write these lanes as $(IV_0, \dots, IV_{n_c-1})$. The IV values are listed in Table 6; their placement in the state is defined in the next subsection.

Table 6: Public IV lanes for the Thunder hash and XOF instances. Each instance uses a distinct c -bit IV for domain separation.

Instance	Index	IV value	Index	IV value
Thunder-512	IV_0	BF6F0F35E3B4B690	IV_5	41E818D4F5A362FC
	IV_1	8E265B9343C855BE	IV_6	694AD1341F066571
	IV_2	246B722CBB56513A	IV_7	9B04C99DC477B767
	IV_3	BC2F395FD06923B4	IV_8	385EC0E6C2C9262C
	IV_4	DD524527A77295D2	—	—
Thunder-768	IV_0	1EE2C8609716E1FD	IV_7	55FBD3148A3A352A
	IV_1	9C67610B0EF5484C	IV_8	900E93CA9A4A9953
	IV_2	1E6D7FF8F050881C	IV_9	9CCCEF7ACDC19758
	IV_3	CC3FA3279D6371F6	IV_{10}	A7A16EAC10689E8C
	IV_4	DB34E00BB8122876	IV_{11}	676B3791EBA6281C
	IV_5	B9839A4AF0CAE3D5	IV_{12}	E1F7C9010E8EC178
	IV_6	DCFA816272A35C8C	—	—
Thunder-1024	IV_0	F40BA520BF2176AF	IV_9	9F76C65C5E81FEAD
	IV_1	6CA062C126D8F82F	IV_{10}	F201204809F567F0
	IV_2	DE0AF08D84B563FD	IV_{11}	717EF7874FDF8324
	IV_3	9DE9EBB8E116C786	IV_{12}	1E74332499E4122A
	IV_4	0E88F98C2AFD6C77	IV_{13}	F8935180A66B22D6
	IV_5	CC6BB54C5AB31102	IV_{14}	948CDBECC8BAE9D8
	IV_6	92814BCC238CCEBA	IV_{15}	2C44DD200F6D93DE
	IV_7	6AF2AB07A720737F	IV_{16}	24503F60BF3CA4C8
	IV_8	95D75D8EE27A1149	—	—
Thunder-XOF-256	IV_0	9E812DA2738FE002	IV_5	4A515E2B8E01C2FF
	IV_1	219B7C0760B0E94D	IV_6	5C0AE2159E01FC77
	IV_2	52156057D367DA1E	IV_7	729855614D7F148C
	IV_3	00A5215DFFDE8879	IV_8	4980F7D439055835
	IV_4	A7883EF829CC1A87	—	—
Thunder-XOF-384	IV_0	F45C092FAC272ECD	IV_7	37F1F11B1F666C79
	IV_1	6663F2D34B981CE9	IV_8	93B2116B09264243
	IV_2	3F8ADB0DCF6E3C57	IV_9	103353E2853DC514
	IV_3	F4E8D96C8C2AA889	IV_{10}	03BF159A8C56AEEA
	IV_4	F63C53A3C826DD36	IV_{11}	63A3C93B73413AB0
	IV_5	6322E49D1D66FD68	IV_{12}	12213C610D4668BF
	IV_6	A69B502F299DF9E8	—	—
Thunder-XOF-512	IV_0	C2884D525C702B64	IV_9	621BC66649423A58
	IV_1	66A72645A0F40A18	IV_{10}	9DE98219698CF8DD
	IV_2	BAB84A614A2350C9	IV_{11}	01C0349676559785
	IV_3	F081647F0AD178DD	IV_{12}	0CB97494B14EA9E1
	IV_4	7B767D98DA24EC2C	IV_{13}	9A48816B41176EB4
	IV_5	3B3C04DD07203847	IV_{14}	7EECDF54AA725D35
	IV_6	FA818BD5EB11B50C	IV_{15}	FFF86452843778BC
	IV_7	D5E7B03035AB65EF	IV_{16}	9AEEEF2B632AA55E
	IV_8	3E1620F081AF14FA	—	—

2.4.2 Message Layout and State Loading

To describe initialization and absorption, we use the linear lane index

$$\text{idx}(x, y) = 5y + x, \quad 0 \leq x, y < 5,$$

consistent with the state map $S[64(5y + x) + z] = D[x][y][z]$ of the permutation specification. Thus, the state lanes are ordered from $S[0, 0]$ to $S[4, 4]$ as $\text{idx}(x, y)$ increases from 0 to 24. We read the 1600-bit string X with $S[0]$ as the least-significant bit, so that the inner part $\text{right}_c(X)$ of the sponge-forward mode, the lower c bits of X , consists of the n_c lanes of smallest linear index.

At any point during the computation, the capacity portion of the state consists of the n_c lanes with the smallest linear indices, while the remaining n_r lanes form the rate portion.

$y = 4$	M_{11}	M_{12}	M_{13}	M_{14}	M_{15}
$y = 3$	M_6	M_7	M_8	M_9	M_{10}
$y = 2$	M_1	M_2	M_3	M_4	M_5
$y = 1$	IV_5	IV_6	IV_7	IV_8	M_0
$y = 0$	IV_0	IV_1	IV_2	IV_3	IV_4
	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$

Figure 5: State initialization and message-word placement for Thunder-512. The capacity portion is initialized with $IV_0, \dots, IV_8$, while the 16 message words occupy the rate portion from M_0 to M_{15} .

During initialization, the IV lanes are loaded into the capacity portion in ascending order of $\text{idx}(x, y)$, whereas all rate lanes are set to zero:

$$S[x, y] = \begin{cases} IV_{\text{idx}(x, y)}, & 0 \leq \text{idx}(x, y) < n_c, \\ 0, & n_c \leq \text{idx}(x, y) < 25. \end{cases}$$

This initialization is performed only once. For all subsequent message blocks, the capacity portion retains the state produced by the previous processing step and is not reloaded with the IV. Message words are mapped exclusively to the rate portion according to the layout defined below.

Each rate block is represented as a sequence of n_r 64-bit words,

$$M' = M_{n_r-1} \| M_{n_r-2} \| \dots \| M_0,$$

where M_{n_r-1} is the most significant word of the block. The message words are associated with the n_r highest-indexed lanes of the state in increasing lane-index order. For $0 \leq i < n_r$, let

$$k_i = 25 - n_r + i, \quad x_i = k_i \bmod 5, \quad y_i = \lfloor k_i/5 \rfloor.$$

Then M_i is associated with lane $S[x_i, y_i]$. Equivalently, M_0 is associated with the lowest-indexed rate lane, whereas M_{n_r-1} is associated with the highest-indexed state lane.

Example (Thunder-512). For Thunder-512, the capacity and rate parameters are $c = 576$ and $r = 1024$, corresponding to $n_c = 9$ capacity lanes and $n_r = 16$ rate lanes. The capacity portion occupies the nine lowest-indexed state lanes,

$$S[0, 0], S[1, 0], \dots, S[4, 0], S[0, 1], S[1, 1], S[2, 1], S[3, 1],$$

which are initialized with $IV_0, IV_1, \dots, IV_8$, respectively. After padding, a message block is represented as $M' = M_{15} \| M_{14} \| \dots \| M_0$. The sixteen message words occupy the sixteen highest-indexed state lanes, with M_{15} mapped to $S[4, 4]$, M_{14} mapped to $S[3, 4]$, and M_0 mapped to $S[4, 1]$. Fig. 5 illustrates the placement of the IV and message words within the state.

2.4.3 Padding and Final Block Processing

Let M be the input message and let $r = 64n_r$ be the rate. The message is processed in successive r -bit blocks. Each complete block is loaded into the rate portion of the state according to the mapping rule defined in the previous subsection. After the absorption of each complete block, a sponge-forward step is performed, that is,

$$X \leftarrow P(X) \oplus (0^r \| S), \quad S = \text{right}_c(X).$$

After all complete r -bit message blocks have been processed, let M_{rem} denote the remaining suffix, where $0 \leq |M_{\text{rem}}| < r$. The remaining suffix is padded by appending one bit 1, followed by enough zero bits so that the padded suffix occupies an integer number of r -bit blocks:

$$\text{pad}(M_{\text{rem}}) = M_{\text{rem}} \| 1 \| 0^{r - ((|M_{\text{rem}}| + 1) \bmod r)}.$$

In this formula, when

$$(|M_{\text{rem}}| + 1) \bmod r = 0,$$

the exponent is interpreted as r . Hence, if the appended bit 1 exactly fills the current rate block, an additional all-zero rate block 0^r is appended. Equivalently,

$$\text{pad}(M_{\text{rem}}) = \begin{cases} M_{\text{rem}} \| 1 \| 0^{r - |M_{\text{rem}}| - 1}, & \text{if } |M_{\text{rem}}| < r - 1, \\ M_{\text{rem}} \| 1 \| 0^r, & \text{if } |M_{\text{rem}}| = r - 1. \end{cases}$$

The padded suffix $\text{pad}(M_{\text{rem}})$ is then split into one or two r -bit blocks and absorbed block by block. Each non-final padded block is loaded into the rate portion of the state and absorbed as

$$X \leftarrow X \oplus (M_i \| 0^c),$$

followed by a sponge-forward step

$$X \leftarrow P(X) \oplus (0^r \| S), \quad S = \text{right}_c(X).$$

Domain separation is applied only during the absorption of the last padded block. Let M_{last} denote the last r -bit block of $\text{pad}(M_{\text{rem}})$. The domain-separation constant is

$$C = 0^{c-1} \| 1.$$

The final padded block is absorbed as

$$X \leftarrow X \oplus (M_{\text{last}} \| C),$$

followed by the final sponge-forward step

$$X \leftarrow P(X) \oplus (0^r \| S), \quad S = \text{right}_c(X).$$

The resulting state is then used for digest extraction.

2.4.4 Digest Extraction

The fixed-output Thunder instances instantiate $\text{Sponge-F}_h^{\text{P}, \text{pd}10^*}$ with $h < c$ for all three parameter sets ($512 < 576$, $768 < 832$, $1024 < 1088$). In this case the squeezing phase consists of a single squeeze with no additional permutation call: the digest is the h -bit output read directly from the inner part of the final state, as in Fig. 4.

For a fixed-output instance with digest length h , let

$$n_d = h/64$$

denote the number of output lanes. The digest is taken from the n_d lowest-indexed lanes of the final state, which lie within the inner part. Using the linear lane index

$$\text{idx}(x, y) = 5y + x,$$

we denote the lane $D[x][y]$ equivalently by

$$D_{\text{idx}(x,y)} = D[x][y].$$

Hence, the extracted digest lanes are

$$D_0, D_1, \dots, D_{n_d-1}.$$

The digest is obtained by serializing these lanes in decreasing lane-index order,

$$\text{digest} = D_{n_d-1} \| D_{n_d-2} \| \dots \| D_0,$$

where each lane is encoded as an eight-byte string in most-significant-byte-first order.

Example (Thunder-512). For Thunder-512, $h = 512$ and therefore $n_d = 8$. The extracted lanes are $D[0][0], D[1][0], D[2][0], D[3][0], D[4][0], D[0][1], D[1][1], D[2][1]$, and the digest is

$$\text{digest} = D[2][1] \| D[1][1] \| D[0][1] \| D[4][0] \| D[3][0] \| D[2][0] \| D[1][0] \| D[0][0].$$

2.5 Extendable-Output Function (XOF)

A feature of Thunder is that the same sponge-with-feed-forward construction supports a variable output length. Instantiating $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ with a chosen capacity gives an extendable-output function. On input (M, ν) it returns the first ν bits of an arbitrarily long output, squeezed r' bits per permutation call from the inner part of the state. The construction is the same as Fig. 2. We provide three instances, Thunder-XOF-256, Thunder-XOF-384, and Thunder-XOF-512. Their parameters are listed in Table 7.

Because the feed-forward lets the squeezing rate r' exceed the absorbing rate r , each squeezing call already produces r' bits of output. The c -bit IV values of the three XOF instances are listed, together with the hash IV s, in Table 6.

2.6 Security Claim

Table 7: The three Thunder extendable-output functions. The security level is the target $\log_2 q$; r and c are the absorbing rate and capacity with $r + c = 1600$; r' is the squeezing rate, the number of output bits produced per permutation call during squeezing. Here q is the number of permutation queries.

Instance	Security ($\log_2 q$)	r	c	r'
Thunder-XOF-256	256	1024	576	1280
Thunder-XOF-384	384	768	832	1152
Thunder-XOF-512	512	512	1088	1024

We state the security claims for the Thunder family as design targets. The claims hold for messages of length up to 2^{64} bits. These targets follow the security of the $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ mode established by Guo et al. [GHJ+25] in the random-permutation model.

Fixed-output hashing. Each Thunder- h instance targets $h/2$ -bit collision security and h -bit preimage security. The feed-forward yields $\min\{c, h\} = h$ -bit preimage security, since $c > h$ for every instance. Concretely, Thunder-512 targets 256-bit collision and 512-bit preimage security, Thunder-768 targets 384-bit collision and 768-bit preimage security, and Thunder-1024 targets 512-bit collision and 1024-bit preimage security. These claims are listed alongside the instance parameters in Table 5.

Extendable output. The extendable-output instances Thunder-XOF-256, Thunder-XOF-384, and Thunder-XOF-512 target a generic security level of $\log_2 q = 256, 384$, and 512 bits, respectively, where q is the number of permutation queries. These levels are given in Table 7.

3 Design Rationale

3.1 The Mode

The sponge construction of Bertoni et al. [BDPVA07, GJMG11] is a popular approach for cryptographic hashing. At a high level, the sponge operates on a state of size b bits, which is split into an inner part of size c bits (the capacity) and an outer part of size r bits (the rate), where $b = c + r$. The sponge consists of an absorbing phase and a squeezing phase. In the absorbing phase, data is compressed into the state r bits at a time, interleaved with an evaluation of a b -bit permutation P . In the squeezing phase, a digest is extracted from the state r' bits at a time (possibly $r' \neq r$, as proposed by [GPP11]), again interleaved with an evaluation of P . The sponge found quick adoption right after its introduction. In particular, Bertoni et al. designed the sponge instance KECCAK, which became the final winner of the NIST SHA-3 competition [D⁺15]. It was quickly acknowledged that the sponge was particularly well-suited for lightweight hashing [BLP⁺08, AHMN10, BKL⁺11, GPP11], and the winner of NIST Lightweight Cryptography competition Ascon-Hash [STMC⁺24] is also a sponge-based algorithm.

To support its security, the designers proved that the sponge construction is indistinguishable from a random oracle up to $2^{c/2}$ adversarial queries [BDPV08]. This implies collision security up to $\min\{2^{c/2}, 2^{h/2}\}$ queries and (second) preimage security up to $\min\{2^{c/2}, 2^h\}$ queries [AMP11], and the preimage security was later improved to $\min\{\max\{2^{h-r'}, 2^{c/2}\}, 2^h\}$ queries by a dedicated proof [LM22]. The latter bound approaches h when r' is very small or $c \approx 2h$. As shown by attacks [BDPVA07, LM22], both the $\min\{\max\{2^{h-r'}, 2^{c/2}\}, 2^h\}$ bit preimage security and the $c/2$ bit second preimage security are roughly tight. Due to this, a sponge-based hashing aiming at h -bit output, $h/2$ bit collision security and $\approx h$ bit (second) preimage security has to set $c \geq 2h$: indeed, this is the choice of the SHA-3 family (SHA3- h with $h \in \{224, 256, 384, 512\}$ defining the output size sets the capacity $c = 2h$). Facing the 1024-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms [NIC25], this constitutes a severe limitation: it means one has to set $c = 2048$ and design a secure permutation with width greater than 2048-bit, which is quite difficult.

It is easy to see that the meet-in-the-middle-style preimage attack in [LM22] crucially relies on a sort of *invertibility* in sponge: given a b -bit intermediate state, one can both invoke P to evaluate forwardly and invoke P^{-1} to evaluate backwardly in the construction. If a *non-invertible* random function is used instead of a random permutation, then Foekens [Foe23] argues that the resulting sponge variant enjoys optimal $\min\{c, h\}$ bit (second) preimage security. By this, a "cryptographically strong" non-invertible function with a width of e.g., 1600 bits, suffices for the required 1024-bit (second) preimage security. However, it is rather non-trivial to design a "cryptographically strong" non-invertible

function.

On the other hand, if we *add feed-forward back* to the permutation-based sponge, then we obtain a sponge variant built upon the *permutation-based Davies-Meyer construction* $\text{DM}^P(X) := P(X) \oplus X$. While the Davies-Meyer construction does not behave like a random function [CDMP05], it is both *non-invertible* and of some cryptographic strength, and might help achieve the 1024-bit (second) preimage security from a e.g., 1600-bit permutation.

This idea is developed into the *sponge-with-feed-forward XOF construction* $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ by Guo et al. [GHJ⁺25], which Thunder adopts as its mode of operation, as illustrated in Fig. 1. For each permutation-call we consider *feeding the c -bit inner part* of the input forward to the output (feeding the outer part forward does not increase security). Moreover, in the squeezing phase the construction produces r' output bits per permutation call. Because the c -bit inner part is fed forward, the squeezing rate r' can exceed the absorbing rate r , while $b - r'$ bits of the state stay hidden, as shown in Fig. 1 (Top). As shown by Andreeva et al. [AMP12], outputting the inner part may cause length-extension attacks. To prevent it, when the last padded message block $\overline{M}[\ell]$ is XORed into the outer part of the state, a constant $C = 0^{c-1}||1$ is simultaneously XORed into the inner part. This borrows ideas from the Merkle-Damgård with permutation construction [HPY12].

Concurrently and independently, Sun et al. [SLZ⁺25] developed the same Davies-Meyer feed-forward idea into their SPONGE-DM and SPONGE-EDM constructions, which achieve comparable h -bit preimage and $\min\{h, c - \log_2 \ell\}$ -bit second-preimage security from a b -bit permutation.

In fact, this idea achieves three gains simultaneously:

- Its collision and (second) preimage security can be reduced to well-defined, simple properties of the underlying permutation;
- It achieves beyond $b/2$ bit (second) preimage security from a b -bit permutation;
- It achieves meaningful quantum collision and (second) preimage security bounds.

As results, let $\ell_{\max}$ be the maximum number of blocks in allowed message inputs. As proven in [GHJ⁺25], when $r = b - c$ is not too small and output length $h \leq c - \log_2 \ell_{\max}$, $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ ensures:

1. $h/2$ bits collision security, h bits preimage security, and $\min\{h, c - \log_2 \ell_{\max}\}$ bits second preimage security;
2. $h/4$ bits quantum collision security, $h/2$ bits quantum preimage security, and $\min\{\frac{h}{2}, \frac{c - \log_2 \ell_{\max}}{2}\}$ bits quantum second preimage security;
3. Approximately $\min\{2^{c/2}, 2^{b-c}\}$ bits indistinguishability security.

The preimage, second preimage and quantum security bounds are much better than those of the standard sponge. By these, to support messages up to 2^{64} bits, we can conclude on the parameter choices as follows:

1. To meet the 512-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms, one needs to set $h = 512$ and $c = 512 + 64 = 576$. When $b \geq 832$, this achieves 256-bit indistinguishability security.
2. To meet the 1024-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms, one need to set $h = 1024$ and $c = 1024 + 64 = 1088$. When $b \geq 1600$, this achieves 512-bit indistinguishability security.
3. To meet the 768-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms, one need to set $h = 768$ and $c = 768 + 64 = 832$. When $b \geq 1216$, this achieves 384-bit indistinguishability security.

Since the output length h is less than c in these cases, $\text{Sponge-FP}^{\text{pd}10^*}$ simply outputs the h bit inner part of the final state, as shown in Fig. 4.

3.2 The Permutation

Choice of state organization. The state of *Thunder-p* is organized as a $5 \times 5 \times 64$ array, following the lane-based structure used in permutation-based hash designs such as KECCAK- $f[1600]$. This organization separates the state into two types of coordinates: the xy -plane, which determines the arrangement of the 25 lanes, and the z -axis, which represents the bit positions inside each 64-bit lane.

This representation is convenient for both software and hardware implementations. On the one hand, each lane can be naturally represented as a 64-bit word. On the other hand, the three-dimensional description makes it clear how different operations act on the state: ρ shifts bits along the z -axis, π permutes lanes in the xy -plane, θ mixes column parities, and ψ applies a nonlinear transformation to 5-bit columns.

Choice of round structure. The round function of *Thunder-p* is composed of five operations:

$$R = \psi \circ \pi \circ \theta \circ \rho \circ \iota.$$

The purpose of this structure is to combine round-dependent asymmetry, bit dispersion, linear diffusion, lane redistribution, and nonlinearity in each round.

The ι operation introduces round-dependent constants and prevents different rounds from being structurally identical. The ρ operation moves bit positions inside each lane and prevents patterns from remaining aligned along the z -axis. The θ operation provides the main linear diffusion by mixing column parity information into the whole state. The π operation rearranges lanes in the xy -plane, so that the output of the linear layer is redistributed before the nonlinear layer. Finally, the ψ operation introduces nonlinearity and prevents the permutation from being described as a purely linear transformation.

Choice of constants. The round constants are introduced to break the symmetry between rounds. Since the ψ operation acts independently for each fixed (x, z) and the θ operation has a highly regular parity-based structure, using no round constants would leave undesirable structural symmetries in the permutation.

In *Thunder-p*, the round constant is added only to lane $(0, 0)$:

$$D'[0][0] = D[0][0] \oplus RC[i_r].$$

This choice keeps the implementation cost low, because only one lane is modified in each round. At the same time, after the following ρ , θ , π , and ψ operations, the effect of the injected constant is propagated to other parts of the state.

The constants are generated by a 5-stage affine LFSR based on the primitive polynomial

$$G(x) = x^5 + x^2 + 1.$$

The affine feedback term avoids the all-zero state being fixed and allows the constant generation process to start from the all-zero initialization. The generated bits are mapped to the bit positions 0, 1, 3, 7, 15 of the round constant, following a sparse constant-injection strategy. This keeps the constant addition lightweight while still ensuring that different rounds are distinguished.

Choice of lane shifts. The ρ operation cyclically shifts each 64-bit lane by a lane-dependent offset $\tau(x, y)$. In the implementation, this operation is realized as a cyclic right rotation. Under the convention that $z = 0$ denotes the least significant bit of a 64-bit lane, the bit positions are shifted to the right by $\tau(x, y)$ positions. Therefore, the ρ operation is specified as

$$D'[x][y][z] = D[x][y][z + \tau(x, y)].$$

As shown in Fig. 6, the ρ operation performs lane-wise cyclic shifts along the z -axis.

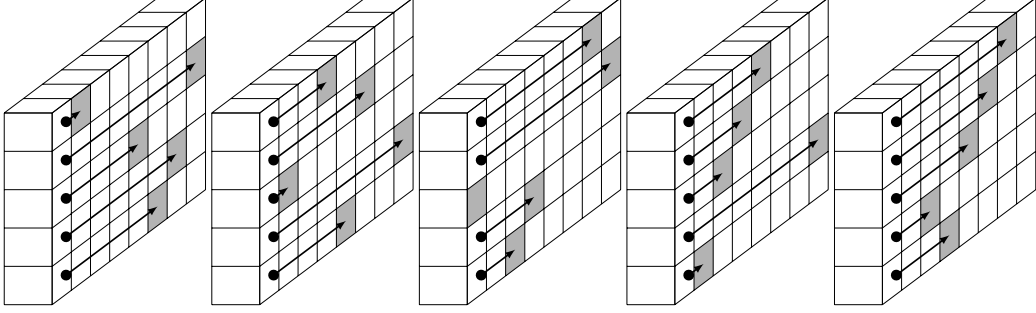


Figure 6: The ρ operation applied to the state of Thunder- p .

The purpose of these offsets is to destroy simple alignment relations along the z -axis. Without such shifts, a difference or mask pattern could remain in the same bit positions across multiple lanes, making it easier to construct structured differential or linear trails.

The offsets $\tau(x, y)$ are selected so that different lanes are shifted by different amounts. This improves the interaction between the z -axis movement introduced by ρ and the column-parity diffusion introduced by θ . Consequently, local patterns in a few bit positions are quickly moved to different longitudinal positions before being mixed by the following linear and nonlinear layers.

Choice of the linear diffusion layer. The θ operation is the main diffusion layer of Thunder- p . As shown in Fig. 7, it computes two column parity planes, folds them with different shifts, and injects the resulting parity information back into the state.

It is designed as a Symmetric Circulant Twin Column Parity Mixer [LRL⁺24], aiming to provide balanced resistance against both differential and linear propagation. A conventional column-parity mixer, as used in Keccak-like designs, is simple and efficient, but its diffusion strength is limited: both its differential and linear branch numbers are only 4. Therefore, such a structure may still allow low-weight differences or masks to remain low-weight after the mixing layer. To improve the diffusion, Thunder- p introduces two column parity planes:

$$P[x][z] = \bigoplus_{y=0}^4 D[x][y][z],$$

and

$$Q[x][z] = \bigoplus_{y=0}^4 D[x][y][z + t_y].$$

The first parity plane P collects the direct column parity, while the second parity plane Q collects a shifted column parity. These two parity planes are then folded as

$$E[x][z] = P[x][z] \oplus P[x-1][z+r],$$

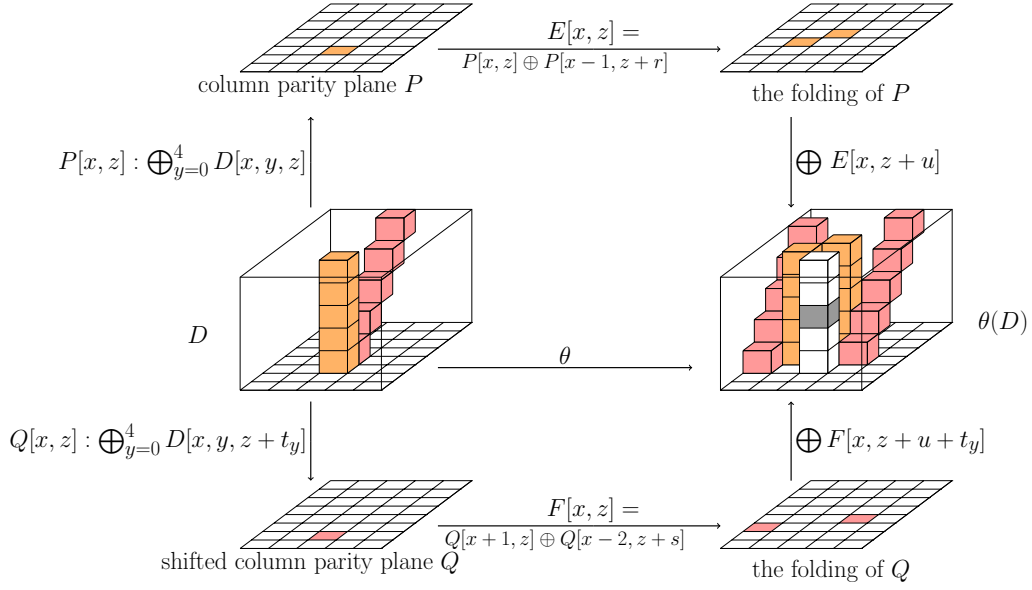


Figure 7: The θ operation applied to the state of Thunder- p .

and

$$F[x][z] = Q[x+1][z] \oplus Q[x-2][z+s].$$

Finally, the two folded values are injected back into the state:

$$D'[x][y][z] = D[x][y][z] \oplus E[x][z+u] \oplus F[x][z+u+t_y].$$

The use of the additional term t_y in both the shifted parity computation and the final injection is the key feature of the symmetric construction. It makes the second parity contribution depend on both the input and output row positions, avoiding the degeneration that may occur in the transpose mapping of a non-symmetric twin column parity mixer. As a result, the differential and linear diffusion properties can be balanced. With the selected parameters, the θ operation achieves differential branch number 12 and linear branch number 12, providing stronger diffusion than the conventional column-parity mixer while preserving a simple XOR-and-rotation structure.

Choice of lane permutation. The π operation permutes the 25 lanes in the xy -plane according to

$$D'[y][2x+3y+2][z] = D[x][y][z].$$

This mapping is affine over the coordinates modulo 5 and is bijective on the set of 25 lanes. Its role is to rearrange the output lanes of the linear layer before the nonlinear layer is applied.

As shown in Fig. 8, the π operation redistributes the 25 lanes within the xy -plane.

The lane permutation prevents the same local structure from being processed in the same geometric position round after round. In particular, it changes the relative positions of lanes after θ , so that the next application of ψ receives inputs from redistributed locations. This improves the interaction between the linear diffusion layer and the nonlinear substitution layer.

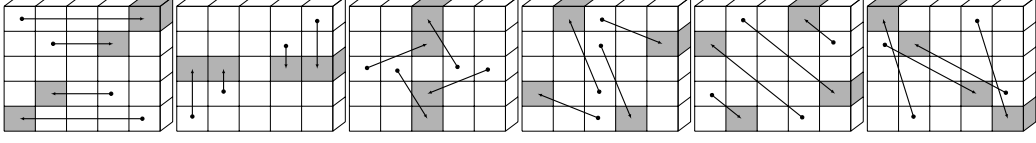


Figure 8: The π operation applied to the state of Thunder- p .

Choice of the nonlinear layer. The ψ operation is the only nonlinear component of Thunder- p . For each fixed pair (x, z) , it acts on five bits along the y -axis:

$$D'[x][y][z] = D[x][y+3][z] \oplus \overline{D[x][y+1][z]} D[x][y+2][z] \overline{D[x][y+4][z]}.$$

As shown in Fig. 9, the ψ operation applies the same 5-bit Sbox independently to each column slice with fixed (x, z) .

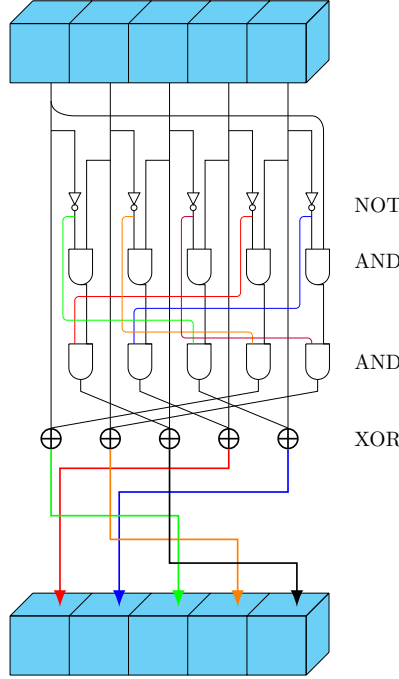


Figure 9: The ψ operation acting on a column of the state of Thunder- p .

This bit-sliced structure is suitable for efficient implementation, since the same 5-bit Sbox is applied independently to many slices of the state.

The nonlinear layer is chosen to provide algebraic and statistical resistance against differential and linear attacks. The algebraic degree of both ψ and ψ^{-1} is 3, and the Sbox has differential uniformity 14 and linearity 24. These properties give a controlled nonlinear behavior while keeping the operation simple.

The direction of ψ is also part of the design. In Thunder- p , ψ acts along the y -axis for fixed (x, z) , whereas the χ operation in KECCAK acts along the x -axis. This choice matches the column-oriented design of the linear layer and allows the diffusion and nonlinear layers to interact through different geometric directions.

Choice of parameter selection criteria. The parameters of Thunder- p are selected according to three main criteria. First, the operations should be simple and efficient on

64-bit platforms. Second, the linear layer should provide strong diffusion and avoid low-weight differential or linear structures. Third, the nonlinear layer should have acceptable differential, linear, and algebraic properties while remaining lightweight.

The shift parameters in ρ and θ are therefore selected to avoid simple invariant patterns along the z -axis and to improve the diffusion of active bits across both the xy -plane and the longitudinal direction. The round constants are selected to distinguish rounds with minimal implementation overhead. Together, these choices make the permutation suitable for a sponge-based hash construction with a 1600-bit internal state.

Choice of rounds. The number of rounds of *Thunder-p* is set to 12 in order to provide a sufficient security margin against the main known structural attacks. Our automated searches show that the best differential and linear trails already decay rapidly within a small number of rounds. And the integral and algebraic-degree analyses also support this round choice. The algebraic degree grows quickly due to the degree-3 nonlinear layer ψ .

4 Security Analysis

4.1 Security analysis of the mode with fixed-output-length

4.1.1 Security Notions

For a conventional fixed-output-length (FOL) hash construction, the primary security requirements are collision and (second) preimage security. Besides, following Coron et al. [CDMP05], one would expect the construction to be indifferentiable from a FOL random oracle. Below we formalize the notions in turn, following the treatment of [GHJ⁺25].

Collision and (Second) Preimage Security This paragraph presents the traditional security definitions for permutation oracle-based fixed-output-length hash functions h^P with h -bit output. First, given an adversary $\mathcal{A}^{P, P^{-1}}$, we define the attack advantage of $\mathcal{A}^{P, P^{-1}}$ against the collision security of h^P as follows:

$$\mathbf{Adv}_h^{\text{coll}}(\mathcal{A}) := \Pr_{P, \varepsilon} [\mathcal{A}^{P, P^{-1}} \Rightarrow (M, M') : M \neq M', h^P(M) = h^P(M')], \quad (2)$$

where the probability is taken over the choice of P (if P has randomness) and the randomness ε of $\mathcal{A}^{P, P^{-1}}$.

Finding a preimage for h^P requires finding $M \in \{0, 1\}^*$ for a challenge image $Z \in \{0, 1\}^h$, such that $h^P(M) = Z$. Given h^P and $\mathcal{A}^{P, P^{-1}}$ and a challenge image $Z \in \{0, 1\}^h$, we define the advantage of $\mathcal{A}^{P, P^{-1}}$ against the Z -preimage security of h^P as follows:

$$\mathbf{Adv}_h^{Z\text{-pre}}(\mathcal{A}) := \Pr_{P, \varepsilon} [\mathcal{A}^{P, P^{-1}}(Z) \Rightarrow M \in \{0, 1\}^*, h^P(M) = Z]. \quad (3)$$

Then, the advantage of $\mathcal{A}^{P, P^{-1}}$ against the *everywhere preimage* security of h^P is defined by taking maximum over all images Z .

$$\mathbf{Adv}_h^{\text{epre}}(\mathcal{A}) := \max_{Z \in \{0, 1\}^h} \left\{ \mathbf{Adv}_h^{Z\text{-pre}}(\mathcal{A}) \right\}. \quad (4)$$

Finding a second preimage for h^P for a challenge message $M \in \{0, 1\}^*$ requires finding $M' \in \{0, 1\}^*$ such that $M' \neq M$ while $h^P(M') = h^P(M)$. Concretely, given $\mathcal{A}^{P, P^{-1}}$ and a challenge message $M \in \{0, 1\}^*$, we define the advantage of $\mathcal{A}^{P, P^{-1}}$ against the M -second preimage security of h^P as follows:

$$\mathbf{Adv}_h^{M\text{-spr}}(\mathcal{A}) := \Pr_{P, \varepsilon} [\mathcal{A}^{P, P^{-1}}(M) \Rightarrow M' \in \{0, 1\}^*, M' \neq M, h^P(M') = h^P(M)]. \quad (5)$$

Typically, second preimage resistance is then defined w.r.t. the bit-length of the challenge message M [RS04, Yas08], which influences concrete security (as observed in [KS05, SLZ⁺25]). Formally, the advantage of $\mathcal{A}^{P, P^{-1}}$ against the everywhere second preimage security of h^P for bl -bit challenge messages is:

$$\text{Adv}_h^{\text{espr}[bl]}(\mathcal{A}) := \max_{M \in \{0,1\}^{bl}} \left\{ \text{Adv}_h^{M\text{-spr}}(\mathcal{A}) \right\}. \quad (6)$$

Indifferentiability of FOL hash. Indifferentiability can only be formalized for random primitive-based constructions. In this respect, let $h^\Pi : \{0,1\}^* \mapsto \{0,1\}^h$ be a FOL hash construction built upon a random permutation Π . We follow [CDMP05] and take a FOL random oracle $\mathbf{RO} : \{0,1\}^* \mapsto \{0,1\}^h$ as the ideal reference. Let $\mathcal{S}^{\mathbf{RO}}$ be a simulator that queries $\mathbf{RO}$ and provides the same interfaces as Π and Π^{-1} . Then, for any distinguisher $\mathcal{D}$, the indifferentiability advantage of $\mathcal{D}$ against h^Π is

$$\text{Adv}_{h^\Pi, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{D}) := \left| \Pr[\mathcal{D}^{\Pi, \Pi^{-1}, h^\Pi} \Rightarrow 1] - \Pr[\mathcal{D}^{\mathcal{S}^{\mathbf{RO}}, P, \mathcal{S}^{\mathbf{RO}}, P^{-1}, \mathbf{RO}} \Rightarrow 1] \right|.$$

Adversarial resource. Hash collision and (second) preimage adversaries would output one or two messages. In reduction proofs, the length of the output message(s) typically affects the reduction complexity and thus concrete security, and should be reflected in adversarial resources. We thus call an oracle-aided adversary $\mathcal{A}^{P, P^{-1}}(q, t, s, \ell_{\max})$ -bounded, if: (i) it makes q queries to P and P^{-1} in total, consumes t computations and s memory, and (ii) after being padded, every message M output by $\mathcal{A}$ has at most $\ell_{\max}$ blocks. In the plain model (i.e., oracle-free setting), we use $(t, s, \ell_{\max})$ -bounded instead. Clearly, $\ell_{\max} = O(t)$, but there may be more stringent restrictions on $\ell_{\max}$ in practice: for example, SHA-256 and SHA-512 have $\ell_{\max} \leq 2^{55} + 1$ and $\ell_{\max} \leq 2^{118} + 1$ respectively [Pub12].

4.1.2 Collision and (Second) Preimage Security Bounds of $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ in the Random Permutation Model

In [GHJ⁺25, Sect. 4.3], the classical security bounds of $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ in the random permutation model are given as the following theorem.

Theorem 1. Assume $b \geq 3$. Consider the construction $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ using a b -bit random permutation Π , output length h , capacity $c \geq h$ and rate r . Then: for any $(q, t, s, \ell_{\max})$ -bounded classical adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$ with $q + 2\ell_{\max} \leq 2^b/2$, $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ has collision, everywhere preimage and everywhere second preimage security bounds as follows:

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{coll}}(\mathcal{A}) \leq \frac{(q + 2\ell_{\max})^2}{2^h} + \frac{2(q + 2\ell_{\max})}{2^h}; \quad (7)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{epre}}(\mathcal{A}) \leq \frac{2(q + \ell_{\max})}{2^h}; \quad (8)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{espr}[bl]}(\mathcal{A}) \leq \frac{\ell^2}{2^h} + \frac{2\ell}{2^c} + \frac{2\ell(q + 2\ell_{\max})}{2^c} + \frac{2(q + 2\ell_{\max})}{2^h}. \quad (9)$$

where $\ell = \max_{M \in \{0,1\}^{bl}} \{|\text{pd}10^*(M)|/r\}$ is the maximal number of blocks in padded challenge messages.

For any $(q, t, s, \ell_{\max})$ -bounded quantum adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$, $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ has quantum collision, quantum everywhere preimage and quantum everywhere second preimage

security bounds as follows:

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{q-coll}}(\mathcal{A}) \leq \frac{4(8q + 16\ell_{\max} + 1)^4}{2^h} + \frac{2(8q + 16\ell_{\max} + 1)^2}{2^c}; \quad (10)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{q-epre}}(\mathcal{A}) \leq \frac{2(8q + 8\ell_{\max} + 1)^2}{2^h}; \quad (11)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{q-espr}[bl]}(\mathcal{A}) \leq \frac{\ell^2}{2^h} + \frac{2\ell}{2^c} + \frac{2\ell(8q + 16\ell_{\max} + 1)^2}{2^c} + \frac{2(8q + 16\ell_{\max} + 1)^2}{2^h}. \quad (12)$$

By these, when $h \leq c - \log_2 \ell_{\max}$, the concrete security bounds achieved by $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ are listed as follows:

1. Classical collision security: $h/2$ bits;
2. Classical preimage security: h bits;
3. Classical second preimage security: h bits;
4. Quantum collision security: $h/4$ bits;
5. Quantum preimage security: $h/2$ bits;
6. Quantum second preimage security: $h/2$ bits.

The bounds are tight, except for the $h/4$ bit quantum collision security bound.

4.2 Security analysis of the mode as an XOF

We view $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ as a (sponge-like) extendable-output function (XOF). According to [GJMG11, Lef23] and NIST FIPS 202 [D⁺15], such a function $\text{XOF}(M, \nu)$ takes a message $M \in \{0, 1\}^*$ and an integer ν as inputs, maps M to a long string and outputs the first ν bits of this string. Such an XOF is typically used for various sophisticated purposes, and one would use indistinguishability from an ideal (sponge-like) XOF, i.e., a VOL random oracle **VRO**, to formalize security for all possible applications. Following [GJMG11, Lef23], we consider a variable-output-length random oracle $\mathbf{VRO} : \{0, 1\}^* \mapsto \{0, 1\}^*$, and consider that the random oracle queried with input-pair $(M, \nu) \in \{0, 1\}^* \times \{\nu_{\min}, \dots, \nu_{\max}\}$ gives $\text{left}_\nu(\mathbf{VRO}(M))$. Let $\mathcal{S}^{\mathbf{VRO}}$ be a simulator that queries **VRO** and provides the same interfaces as Π and Π^{-1} . Then, for any distinguisher $\mathcal{D}$, the indistinguishability advantage of $\mathcal{D}$ against XOF^Π is

$$\text{Adv}_{\text{XOF}^\Pi, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{D}) := \left| \Pr[\mathcal{D}^{\Pi, \Pi^{-1}, \text{XOF}^\Pi} \Rightarrow 1] - \Pr[\mathcal{D}^{\mathcal{S}^{\mathbf{VRO}}, \mathbf{P}, \mathcal{S}^{\mathbf{VRO}}, \mathbf{P}^{-1}, \mathbf{VRO}} \Rightarrow 1] \right|.$$

Indistinguishability Security Bounds of $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ and $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ in the Random Permutation Model. Following Bertoni et al. [BDPV08], we measure the cost of a construction query based on the number of permutation evaluations required in the real world to produce the output. Concretely, each call to $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}(M, \nu)$ with ℓ -block $\bar{M}$ has a cost of ℓ . Therefore, if the adversary makes p queries to Π and Π^{-1} and q queries to $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ and the padded messages have at most σ blocks in total (under our parsing role), then the *total oracle query cost* is at most $Q \leq p + \sigma$, which is the Q value introduced in Theorem 2.

With the above, in [GHJ⁺25, Sect. 6], the indistinguishability bound of $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ in the random permutation model is given as the following theorem.

Theorem 2. Assume $c \geq 3$ and $Q \geq 8$. Then, there exists a simulator $\mathcal{S}^{\mathbf{VRO}}$ such that the following holds for any differentiator $\mathcal{D}$ with total number of oracle query costs Q :

$$\mathbf{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{D}) \leq \frac{1}{2^c} + \frac{(4c+2)Q^2}{2^b} + \frac{12Q+14Q^2}{2^c} + \frac{5Q \log_2 Q}{2^{b-c}}.$$

Moreover, $\mathcal{S}^{\mathbf{VRO}}$ makes at most Q queries to $\mathbf{VRO}$.

Roughly speaking, indistinguishability implies that $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ satisfies all security properties that can be defined using single-stage games [RSS11, DGHM13] and can be achieved by the VOL random oracle $\mathbf{VRO}$, up to the indistinguishability bound of $\text{Sponge-F}^{\Pi, \text{pd}10^*}$. This includes the well-known security against multi-collision attacks and length-extension attacks. But the concrete security may be limited by $\mathbf{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{D})$. In detail, according to the concrete version of the indistinguishability composition theorem [10, Theorem 3.1], for any (q, t, s) -bounded classical collision adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$ against $\text{Sponge-F}^{\Pi, \text{pd}10^*}$, there exists a collision adversary $\mathcal{B}^{\mathbf{VRO}}$ against $\mathbf{VRO}$, such that

$$\begin{aligned} & \Pr[\mathcal{A}^{\Pi, \Pi^{-1}}(\nu) \Rightarrow (M_1, M_2) : \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_1, \nu) = \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_2, \nu)] \\ & \leq \mathbf{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{A}) \\ & \quad + \Pr[\mathcal{B}^{\mathbf{VRO}}(\nu) \Rightarrow (M_1, M_2) : \mathbf{VRO}(M_1, \nu) = \mathbf{VRO}(M_2, \nu)]. \end{aligned}$$

The adversary $\mathcal{B}^{\mathbf{VRO}}$ is constructed by “combining” $\mathcal{A}^{\Pi, \Pi^{-1}}$ with the simulator $\mathcal{S}$. Therefore, $\mathcal{B}^{\mathbf{VRO}}$ makes at most q queries to $\mathcal{RO}$, and this means the probability that $\mathcal{B}^{\mathbf{VRO}}$ finds distinct messages M_1, M_2 with $\mathbf{VRO}(M_1, \nu) = \mathbf{VRO}(M_2, \nu)$ is $q^2/2^\nu$. By these,

$$\begin{aligned} & \Pr[\mathcal{A}^{\Pi, \Pi^{-1}}(\nu) \Rightarrow (M_1, M_2) : \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_1, \nu) = \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_2, \nu)] \\ & \leq \frac{q^2}{2^\nu} + \frac{1}{2^c} + \frac{(4c+2)Q^2}{2^b} + \frac{12Q+14Q^2}{2^c} + \frac{5Q \log_2 Q}{2^{b-c}}. \end{aligned}$$

When $\nu < c$ and $b - c \geq c/2$, it indicates collision security up to $2^{\nu/2}$ queries; when $\nu \geq c$ and $b - c \geq c/2$, it indicates collision security up to $2^{c/2}$ queries. The latter is comparable with that given in Eq. (7). Note that when $\nu \geq c$, collision security does not increase with ν .

Similarly, for any $(q, t, s, \ell_{\max})$ -bounded classical preimage adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$ against $\text{Sponge-F}^{\Pi, \text{pd}10^*}$, there exists a collision adversary $\mathcal{B}^{\mathbf{VRO}}$ against $\mathbf{VRO}$, such that

$$\begin{aligned} & \Pr[\mathcal{A}^{\Pi, \Pi^{-1}}(Z, \nu) \Rightarrow M : \text{Sponge-F}^{\Pi, \text{pd}10^*}(M, \nu) = Z] \\ & \leq \mathbf{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{A}) + \Pr[\mathcal{B}^{\mathbf{VRO}} \Rightarrow M : \mathbf{VRO}(M, \nu) = Z] \\ & \leq \frac{q}{2^\nu} + \frac{1}{2^c} + \frac{(4c+2)Q^2}{2^b} + \frac{12Q+14Q^2}{2^c} + \frac{5Q \log_2 Q}{2^{b-c}}. \end{aligned}$$

When $\nu < c/2$ and $b - c \geq c/2$, it indicates preimage security up to 2^ν queries; when $\nu \geq c/2$ and $b - c \geq c/2$, it indicates preimage security up to $2^{c/2}$ queries. The latter is weaker than that given in Eq. (8), since the indistinguishability security of $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ is limited by roughly $c/2$ bits: this resembles the indistinguishable Merkle-Damgård variants [CDMP05, KK06].

Concrete multi-collision security bounds can be derived similarly, but for most parameters, $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ only ensures $c/2$ -bit multi-collision security. This also resembles the indistinguishable Merkle-Damgård variants [Jou04].

Finally, fixing ν to the fixed output length h yielding indistinguishability bound and interpretations for the FOL construction $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$, which are actually similar to the XOF $\text{Sponge-F}^{\Pi, \text{pd}10^*}$.

4.3 Security analysis of Thunder- p

4.3.1 Full diffusion property

To evaluate the diffusion speed of Thunder- p , we analyze the structural bit-dependency relation of the reduced-round permutation. Specifically, for each input bit, we count the number of output bits that can be affected by it, which is referred to as the forward coverage. A reduced-round permutation is considered to reach full diffusion if every input bit can affect all 1600 output bits. Table 8 summarizes the forward diffusion behavior of Thunder- p . After one round, each input bit affects 84 output bits. After two rounds, the minimum forward coverage increases to 1553 bits, showing that the dependency relation is already close to covering the full state. After three rounds, the minimum forward coverage reaches 1600, and all 1600 input bits can affect all 1600 output bits. Therefore, Thunder- p achieves full structural diffusion after 3 rounds in the forward direction.

Table 8: Forward structural full-diffusion behavior of Thunder- p .

Rounds	Min. forward coverage	Fully diffused input bits
1	84	0
2	1553	0
3	1600	1600

4.3.2 Differential cryptanalysis

In this section, we evaluate the differential propagation properties of Thunder- p . Since the mixing layer of Thunder- p has differential branch number 12, any nonzero input difference must induce at least 12 active columns across the input and output of the mixing layer. Therefore, the low-weight differential trails exploited in the KECCAK-f search do not exist for Thunder- p . Combined with the differential uniformity 14 of the ψ transformation, it follows that, in the two-round case, Thunder- p cannot have any differential trail with probability higher than

$$(14/32)^{12} = 2^{-14.31}.$$

This bound is tight, as demonstrated by the optimal two-round differential trail given in Table 21 in the appendix.

For a larger number of rounds, to obtain security arguments, we construct a differential propagation model of Thunder- p using the constraint modelling language MiniZinc [NSB⁺07], and employ the hybrid constraint solver Chuffed [CSS⁺18] to automatically search for the number of active ψ instances and differential characteristics.

Construction of the Objective Function. In differential analysis, the key metric is the propagation probability of a differential state through nonlinear components. To transform the problem of maximizing the product of differential probabilities into a minimization problem that can be efficiently handled by CP solvers, we introduce the negative logarithm of the differential probability as a weight variable.

In our MiniZinc model, the internal state is represented as a three-dimensional bit array indexed by column, row, and lane. More precisely, for each round n , the state is indexed by (x, y, z) , where $x \in 0, \dots, 4$ denotes the column coordinate, $y \in 0, \dots, 4$ denotes the row coordinate, and $z \in 0, \dots, 63$ denotes the lane position. The nonlinear transformation ψ is applied to each 5-bit column slice with fixed column x and fixed lane position z , i.e., to the five bits over all rows:

$$(D[x][0][z], D[x][1][z], D[x][2][z], D[x][3][z], D[x][4][z]).$$

Therefore, each ψ instance is uniquely indexed by the pair (x, z) .

Accordingly, we define $w_{n,x,z}$ as the weight variable corresponding to the propagation from the input difference d_{in} to the output difference d_{out} through the ψ transformation at column x and lane position z in the n -th round. Essentially, this weight is the logarithmic mapping of the differential probability $DP(d_{in}, d_{out})$:

$$w_{n,x,z}(d_{in}, d_{out}) = -\log_2 \left(\frac{a \in \mathbb{F}_2^5 \mid \psi(a) \oplus \psi(a \oplus d_{in}) = d_{out}}{2^5} \right).$$

Under this definition, the total probability of an r -round differential trail is given by the product of the differential probabilities of all active ψ instances. After mapping the probabilities to the logarithmic domain, maximizing this product is equivalent to minimizing the sum of the corresponding weight variables. Therefore, the objective function of the CP solver is set to minimize the total weight over all rounds, columns, and lane positions:

$$\text{Minimize} \quad \sum_{n=0}^{r-1} \sum_{x=0}^4 \sum_{z=0}^{63} w_{n,x,z}.$$

In the MiniZinc implementation, these weights are stored as integer-scaled probability weights in the array `prb`, and the objective value is defined as their sum.

State Representation and Forward Propagation. According to the internal structure of *Thunder-p*, we define several three-dimensional Boolean variable arrays for each round to precisely track the propagation of differential states through each operation. For the n -th round, the initial input difference state is denoted by $S_{n,1}$, corresponding to the variable `statestart`. The state after the nonlinear layer ψ is denoted by $S_{n,2}$, corresponding to `stateend`. Then, the lane shift operation ρ is applied, producing the state $S_{n,3}$, corresponding to `stateeast`.

After that, the state passes through the symmetric circulant column parity mixing layer θ . In the MiniZinc model, this layer is described by a set of intermediate variables. First, the parity terms P and Q are computed from the shifted state. Then, the auxiliary terms E and F are derived from P and Q with additional rotations. Finally, the output of the mixing layer is obtained by xoring the shifted state with the rotated E and F terms. This output state is denoted by $S_{n,4}$, corresponding to `statestart0`. The round transition is completed by the coordinate permutation π , which maps $S_{n,4}$ to the initial input state of the next round, namely $S_{n+1,1}$. Hence, the boundary constraint between two consecutive rounds is realized by the π mapping from `statestart0` to `statestart` of the next round.

Modeling of the ψ Transformation. We adopt a table-based method to model the differential propagation probability through the nonlinear layer ψ . For each fixed column x and lane position z , the input and output differences of ψ are both 5-bit values formed by the five row bits. First, we compute the differential distribution table of ψ and extract all input-output difference pairs (d_{in}, d_{out}) with nonzero differential probability, together with their corresponding differential propagation weight w .

Then, the five bits of the input difference, the five bits of the output difference, and the corresponding weight are concatenated into an 11-tuple. All valid 11-tuples are collected into a constant transition table. In MiniZinc, a global table constraint is imposed on every pair (x, z) in every round, forcing the differential propagation through each ψ instance to be valid.

Modeling of the Linear Layer. The modeling of the linear layer consists of the lane shift operation ρ , the symmetric circulant column parity mixing layer θ , and the coordinate permutation π . The operation ρ is implemented by rotating the lane indices according to the rotation parameters associated with each column and row.

For the θ operation, the model first computes two parity-dependent terms P and Q . The term P is obtained by xoring the five row bits for each fixed column and lane position,

while Q is computed similarly after applying row-dependent lane rotations. The auxiliary terms E and F are then derived from P and Q using column shifts and lane rotations. Finally, the output of θ is obtained by xoring the shifted state with the corresponding rotated E and F terms. All xor relations are encoded by parity constraints, namely by requiring the sum of all input and output variables involved in each xor equation to be even.

The coordinate permutation π is modeled by directly re-indexing the column and row coordinates, while keeping the lane coordinate unchanged. This establishes the connection between the output state of the current round and the input state of the next round.

Search Strategy.

If the basic model described above is directly applied, the search space of the solver grows exponentially, and the solver often only finds suboptimal trails with relatively low differential probabilities. To reduce the search complexity, we further exploit a special class of states of the mixing layer. These states are called the kernel states of the θ operation.

Definition 1 (Kernel space of the θ operation in *Thunder-p*). Let D be the input state before the θ operation. If, for all x, y , and z , the parity-dependent correction terms satisfy

$$E[x][z + u] \oplus F[x][z + u + t_y] = 0,$$

then D is called a kernel state of the θ operation. The set of all such states is called the kernel space of the θ operation.

For any state in this kernel space, the correction introduced by the mixing layer is cancelled. Therefore, the input and output of the θ operation are identical, and the mixing layer behaves as the identity mapping on these states. In other words, the diffusion effect of the mixing layer is neutralized in the kernel space, where low-diffusion states may exist and may provide favorable starting points for constructing high-probability differential trails.

In our model, we force the first-round parity variables of the mixing layer to satisfy

$$P = 0, \quad Q = 0.$$

Since the correction terms E and F are determined by the parity variables P and Q , the constraints $P = 0$ and $Q = 0$ directly make the parity-dependent correction vanish. Hence, the constrained states naturally belong to the kernel space of the θ operation. Under this setting, the first-round mixing layer acts as an identity mapping, and the search space is restricted to the kernel space of *Thunder-p*. This restriction significantly reduces the search complexity and guides the solver toward low-diffusion trails with relatively high differential probabilities. To show that high-probability trails are more likely to appear in the kernel space, we also perform a comparison search in the out-of-kernel state space. In this setting, the input and output of the mixing layer are required to be different in each round, so that the searched trails do not remain in the kernel of the mixing layer. In the implementation, the search is further guided by the activation variables of the ψ instances and the probability-weight variables, and the CP solver minimizes the total objective value under these constraints. The results are shown in Table 9.

Table 9: Known best numbers of active differential columns and differential propagation probabilities of *Thunder-p* for different numbers of rounds

Rounds	Active Columns		Differential Probability	
	In Kernel	Out Kernel	In Kernel	Out Kernel
2	12	22	$2^{-14.31}$	$2^{-26.24}$
3	106	218	$2^{-144.06}$	$2^{-342.58}$
4	401	516	$2^{-781.83}$	$2^{-997.73}$
5	694	805	$2^{-1394.95}$	$2^{-1610.08}$

4.3.3 Linear cryptanalysis

First, since there exists no state with linear branch number smaller than 12 in the kernel space of *Thunder-p*, the number of active columns of *Thunder-p* after two rounds must be at least 12. Combined with the maximum absolute correlation value $24/32$ of the ψ transformation, it follows that, in the two-round case, *Thunder-p* cannot have any linear trail with squared correlation higher than

$$(24/32)^{2 \times 12} = 2^{-9.96}.$$

This bound is tight, as demonstrated by the optimal two-round linear trail given in Table 22 in the appendix.

For a larger number of rounds, similar to the differential analysis, we adopt a similar modeling and search strategy for the search. Specifically, we use a comparable constraint model and search space. However, we observe that imposing only the constraints $P = 0$ and $Q = 0$ leads to much worse search results than in the differential case. Therefore, we further restrict our attention to a small subset of the kernel space. In particular, we only consider the states for which the numbers of active bits before and after the first θ operation are both equal to 6, i.e., the states attaining branch number 12. The results are shown in Table 10.

Table 10: Known best numbers of active linear columns and squared correlations of linear trails of *Thunder-p* for different numbers of rounds

Rounds	Active Columns		Squared Correlation	
	In Kernel	Out Kernel	In Kernel	Out Kernel
2	12	22	$2^{-9.96}$	$2^{-18.26}$
3	105	234	$2^{-93.50}$	$2^{-278.47}$
4	394	531	$2^{-637.87}$	$2^{-816.42}$
5	712	831	$2^{-1166.19}$	$2^{-1363.74}$

4.3.4 Impossible Differential and Zero-Correlation

For *Thunder-p*, our diffusion analysis shows that full diffusion is achieved after 3 rounds. In general, the number of rounds covered by diffusion-based miss-in-the-middle impossible differential and zero-correlation distinguishers can be roughly estimated as twice the number of rounds required for full diffusion. Therefore, for *Thunder-p*, such distinguishers are expected to cover approximately

$$2 \times 3 = 6$$

rounds. Since this estimate is significantly below the full-round configuration of *Thunder-p*, impossible differential and zero-correlation attacks based on diffusion arguments are not considered a practical threat to *Thunder-p*.

4.3.5 Higher-order differential cryptanalysis

Higher-order differential attack [Knu94, Lai94] exploits the fact that given a vectorial Boolean function $f: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ of degree δ , then for any affine subspace $\mathcal{V} \subseteq \mathbb{F}_2^n$ of dimension at least $\delta + 1$, we have

$$\bigoplus_{x \in \mathcal{V}} F(x) = 0.$$

As described in [BCC11], there exists an upper bound for the algebraic degree of iterated permutations for SPN schemes. By combining this bound with the trivial bound 3^r , we get the bound in Table 11. The algebraic degree of the inverse permutation is the same.

Table 11: Upper bounds on the algebraic degree of the (inverse) Thunder- p permutation.

Rounds	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Degree	3	9	27	81	243	729	1309	1503	1567	1589	1596	1598	1599	1599

4.3.6 Integral cryptanalysis

The core idea of integral cryptanalysis [DKR97, KW02] is to construct a structured set of input plaintexts such that the sum of corresponding outputs at some certain positions exhibits integral properties, such as zero-sum. Using the path search algorithm proposed by Todo et al. [Tod15], we construct the integral distinguishers for Thunder- p permutation. The results are reported in Table 12.

We do not explicitly consider weak distinguishers such as zero-sum distinguishers, as the non-ideal properties of the permutation do not seem to affect the security of Thunder- p .

Table 12: Integral distinguishers of Thunder- p permutation. D represents the data complexity in $\log_2$.

Rounds	1	2	3	4	5	6	7	8	9	10	11	12
D	4	5	10	30	84	245	730	1310	1505	1570	1590	1599

4.3.7 Differential-linear cryptanalysis

We assess the resistance of Thunder- p against differential-linear (DL) cryptanalysis with the round-based correlation approximation [HNW26].

Configuration. An R -round DL distinguisher of Thunder- p covers the segment that begins at the input difference of a ψ layer and ends at the output mask of a ψ layer,

$$\psi \circ L \circ \psi \circ L \circ \dots \circ L \circ \psi,$$

with R applications of the nonlinear layer ψ and $R - 1$ applications of the linear layer $L = \pi \circ \theta \circ \rho$. The round constant ι does not affect the difference and mask propagation and is omitted. We follow the three-part split $E = E_l \circ E_m \circ E_d$ of [HNW26], where the front E_d spans r_d rounds, the middle E_m spans r_m rounds, and the linear tail E_l spans r_l rounds with $r_d + r_m + r_l = R$. The differential transition of E_d is evaluated exactly through the differential distribution table of ψ , and the DL correlation of E_m is estimated round by round under the geometric approach. We set $r_d = 1$ and $r_l = 0$, so the input difference Δ enters one exact differential round and the output mask Λ is read directly at the output of E_m .

Search and results. For each number of rounds R we search over all input differences Δ and output masks Λ that activate a single ψ instance, and report the pair maximizing the absolute DL correlation $|\text{Cor}(\Delta, \Lambda)|$. An input difference or output mask of Hamming weight one is written as the coordinate (x, y, z) of its active state bit $D[x][y][z]$. Table 13 lists the best distinguishers.

Table 13: Best single-active differential-linear distinguishers of the **Thunder- p** permutation under the round-based approximation with $r_d = 1$ and $r_l = 0$. The round-based column is the approximation of [HNW26], and the experimental column is the correlation measured on the real permutation by experimental sampling.

Rounds R	Input difference Δ	Output mask Λ	$\log_2 \text{Cor} $	
			Round-based	Experimental
2	(0, 4, 0)	(0, 4, 0)	0	0
3	(4, 0, 0)	(2, 3, 46)	-0.13	-0.13
4	(0, 1, 3)	(0, 0, 55)	-16.54	-16.00
5	(3, 1, 14)	(1, 1, 46)	-559.95	-

The two-round distinguisher has correlation exactly 1 and the three-round distinguisher has correlation 0.913. We confirmed both against the real permutation by experimental evaluation, and a 2^{40} -sample measurement gives the four-round correlation as $2^{-16.00}$, slightly above the round-based estimate of $2^{-16.54}$. From four rounds onward the correlation drops sharply. At five rounds the best single-active correlation is $2^{-559.95}$, giving a data complexity of about 2^{1120} , which already exceeds the 1024-bit security. For $R \geq 6$ the best single-active estimate falls below 2^{-1074} . Thus **Thunder- p** offers full resistance to single-active differential-linear distinguishers from five rounds onward.

4.4 Security analysis of the hash function

4.4.1 Collision Security under Differential Trail Attack

This subsection analyzes a two-round collision trail of **Thunder-512** in a controllable-IV setting. This setting assumes that the adversary is allowed to control the initial state difference, including the IV part. Therefore, the collision condition does not have to be satisfied only by message differences; instead, the input difference can be chosen so that, after the linear part of the first round, the state difference enters a prescribed low-weight nonlinear trail.

Let one round of **Thunder- p** be written as

$$R = \psi \circ \pi \circ \theta \circ \rho \circ \iota.$$

Since the round constant addition (ι) is independent of the input state, it has no effect on differences. Hence, for differential propagation, the linear part before the nonlinear layer can be regarded as

$$L = \pi \circ \theta \circ \rho.$$

In the controllable-IV model, we choose the initial difference (ΔX) so that the difference after (L) satisfies the prescribed input condition of the first nonlinear layer. After two rounds, the difference on the output hash positions is constrained to be zero. Therefore, the two-round output condition can be written as

$$\Delta Y_h = 0,$$

where (ΔY_h) denotes the difference on the hash-output part of the state.

This condition does not by itself give a deterministic collision. More precisely, it defines a two-round collision differential trail: for a pair of initial states with the chosen controllable-IV difference, the difference on the hash-output positions is forced to zero only if the pair actually follows the specified nonlinear differential transitions through the two rounds. Thus, in the controllable-IV model, the trail gives a probabilistic two-round collision trail for the reduced version of Thunder-512, rather than a deterministic collision for all pairs satisfying the input difference.

The following trail was found by the differential search model. The coordinate (x, y, z) denotes the active bit at sub-state or column index x , row index y , and bit position z . The state "Before ψ " denotes the input difference of the nonlinear layer, and "After ψ " denotes the output difference of the nonlinear layer.

Table 14: A two-round collision trail of Thunder-512 under controllable IV

Round	Stage	Active bit positions (x, y, z)	Active S-boxes	Weight
0	Before ψ	$\{(2, 0, 21), (2, 0, 54), (2, 1, 22), (2, 1, 51), (2, 3, 34), (2, 3, 38)\}$	6	$6 \log_2(32/14)$
0	After ψ	$\{(2, 0, 34), (2, 0, 38), (2, 2, 21), (2, 2, 54), (2, 3, 22), (2, 3, 51)\}$	6	—
1	Before ψ	$\{(0, 1, 21), (0, 1, 25), (2, 2, 21), (2, 2, 54), (3, 0, 25), (3, 0, 54)\}$	6	$6 \log_2(32/14)$
1	After ψ	$\{(0, 3, 21), (0, 3, 25), (2, 4, 21), (2, 4, 54), (3, 2, 25), (3, 2, 54)\}$	6	—
Total			12	14.31

Equivalently, the real trail weight is approximately

$$12 \times 1.1926 = 14.31.$$

The trail shows that, when the IV difference is controllable, one can choose the input difference such that the difference after $(\pi \circ \theta \circ \rho \circ \iota)$ enters the low-weight nonlinear transitions listed in Table 14. If these nonlinear transitions are followed, then after the second round the difference on the hash-output positions becomes zero. Therefore, the two-round reduced Thunder-512 admits a probabilistic collision differential trail under the controllable-IV assumption, rather than a deterministic collision for every pair satisfying the initial difference.

It should be emphasized that this is a reduced-round and controllable-IV observation. It does not directly imply a practical collision attack on the full 12-round Thunder-512 with a fixed IV, because in the standard hash setting the IV is fixed and the attacker can only control the absorbed message part. For the fixed-IV setting, as well as for Thunder-768 and Thunder-1024, we did not find any exploitable high-probability differential trail that can be used to mount a collision attack.

4.4.2 Randomness Test

To evaluate the statistical randomness of the output sequences generated by Thunder, we apply the NIST SP 800-22 statistical test suite to three instances: Thunder-512, Thunder-768, and Thunder-1024. The test is conducted under a counter-based input generation setting. Specifically, the initial counter is set to 2^{32} . For the i -th input message, we define

$$M_i = \text{uint64_be}(2^{32} + i), \quad i = 0, 1, 2, \dots,$$

where $\text{uint64_be}(\cdot)$ denotes the 64-bit big-endian encoding of the counter value. The corresponding hash outputs are computed independently and concatenated into a continuous bit stream:

$$H(M_0) \| H(M_1) \| H(M_2) \| \dots$$

This bit stream is further divided into fixed-length binary sequences and used as the input to the NIST SP 800-22 statistical test suite.

In this experiment, the length of each test sequence is set to

$$\text{BITS_PER_SEQUENCE} = 1,000,000,$$

and 100 test sequences are generated for each Thunder instance, namely

$$\text{SEQUENCE_COUNT} = 100.$$

The test input files are stored in ASCII format, where each bit is represented by the character 0 or 1. Within each digest byte, bits are written in most-significant-bit-first order. Therefore, the input mode of the NIST STS is set to ASCII. All statistical tests are executed with the default parameters, and the significance level is set to $\alpha = 0.01$.

The NIST STS results are evaluated from two aspects: the uniformity of the resulting p -values and the proportion of passing sequences. For a sample size of 100 sequences, the minimum acceptable pass rate is approximately 96/100 for all tests except the Random Excursions and Random Excursions Variant tests. Since the Random Excursions tests are only applicable to sequences satisfying the required cycle condition, their effective sample sizes may be smaller than 100. The corresponding minimum pass rates are therefore determined by the test suite according to the actual number of valid sequences.

The summarized results are shown in Table 15. It can be observed that all test items for Thunder-512, Thunder-768, and Thunder-1024 satisfy the corresponding pass-rate requirements. In addition, the minimum uniformity p -values of the three instances are all greater than 10^{-4} , indicating that the obtained p -values do not exhibit obvious non-uniform behavior. Therefore, under the above testing conditions, no observable statistical deviation from randomness is detected by the NIST SP 800-22 statistical test suite for the tested Thunder output sequences.

Table 15: Summary of NIST SP 800-22 randomness test results for Thunder

Instance	Number of sequences	Sequence length	Minimum p -value	Minimum observed pass rate
Thunder-512	100	10^6 bits	0.008266	96/100; 58/61 for RE/RV
Thunder-768	100	10^6 bits	0.006196	96/100; 57/59 for RE/RV
Thunder-1024	100	10^6 bits	0.001588	96/100; 66/68 for RE/RV

Here, RE/RV denotes the Random Excursions and Random Excursions Variant tests. For the RE/RV tests, the effective sample sizes are 61, 59, and 68 for Thunder-512, Thunder-768, and Thunder-1024, respectively. The corresponding minimum acceptable pass rates reported by the test suite are 58/61, 56/59, and 64/68, respectively. The observed minimum RE/RV pass rates are 58/61, 57/59, and 66/68, respectively. In particular, the minimum observed RE/RV pass rate of Thunder-512 is 58/61, which exactly meets the corresponding minimum acceptable pass-rate requirement, while the observed values for Thunder-768 and Thunder-1024 are above their corresponding requirements.

It should be noted that statistical testing only indicates that no significant statistical deviation is observed under the given test scale and test setting; it does not constitute a mathematical proof of true randomness.

4.4.3 Collision Security under Linearized Differential Trail Attack

This subsection assesses the collision resistance of the Thunder family through a linearized differential trail cryptanalytic method [AHY23]. The core objective of this attack is to

construct an R-round differential trail satisfying boundary conditions consistent with the sponge-with-feed-forward framework: the input capacity difference $\Delta X_c = 0$, since message data is absorbed only into the rate region, and the output digest difference $\Delta Y_h = 0$, which guarantees identical hash digests. The full R-round trail is split into a two-round connector and an (R-2)-round colliding trail. The first two rounds are linearized to derive a linear subspace of valid rate inputs that satisfy the differential propagation rules of the connector, and the adversary exhausts all vectors within this subspace to test whether they follow the colliding differential path with trail probability p . The attack is computationally feasible only if the remaining free degrees of freedom after imposing all linear constraints satisfy $d_{\text{remain}} > -\log_2 p$, where $-\log_2 p$ quantifies the minimal search complexity required for the colliding trail.

The permutation consists of a 1600-bit state split into 320 independent 5-bit S-boxes, and preceding linear layers propagate differential constraints across the entire state, which mandates full linearization of all 320 S-boxes. Linear subspaces can be found when input and output differences of S-boxes are fixed.

Observation 1. *Given a 5-bit input difference δ_{in} and a 5-bit output difference δ_{out} such that $DDT(\delta_{in}, \delta_{out}) \neq 0$, and the solution set is V , we have:*

- a . if $DDT(\delta_{in}, \delta_{out}) = 2$, then V is a 1-dimensional linear subspace.*
- b . if $DDT(\delta_{in}, \delta_{out}) = 4$, then V is a 2-dimensional linear subspace.*
- c . if $DDT(\delta_{in}, \delta_{out}) = 6$ or 8 , then V is at most a 2-dimensional linear subspace with extra linear constraints introduced.*
- d . if $DDT(\delta_{in}, \delta_{out}) = 14$, then V is at most a 3-dimensional linear subspace with extra linear constraints introduced.*

Following the assumption that each active S-box with fixed input and output differentials imposes at least three independent linear constraints consuming three degrees of freedom, the first round incurs a minimal freedom consumption of $C_1 = 3 \times 320 = 960$. Let a denote the total number of active S-boxes in the second round; the second round introduces additional constraints cost $C_2 = 3a$. Given r as the total available free degrees of freedom corresponding to the rate bit length of each hash instance, the remaining usable subspace dimension is formalized as $d_{\text{remain}} = r - (960 + 3a)$. If $d_{\text{remain}} \leq -\log_2 p$, the size of input space can not satisfy the colliding trail. For Thunder-1024 with $r = 512$ and Thunder-768 with $r = 768$, d_{remain} fail to satisfy the necessary dimensional condition. Even for Thunder-512 with $r = 1024$, the remaining degrees of freedom are insufficient to satisfy the dimensional requirement of the linear subspace that yields colliding message pairs.

4.4.4 Collision Attack based on Rebound Attack

The rebound attack, introduced by Mendel et al. [MRST09], writes a primitive as $P = E_F \circ E_I \circ E_B$ and starts the search in the middle. The *inbound* phase uses the freedom of the internal state to satisfy the dense middle part E_I across a nonlinear layer by a meet-in-the-middle over the differential distribution table (DDT), while the two *outbound* parts E_B and E_F are only fulfilled probabilistically, with probabilities p_B and p_F . The attack is most effective against strongly aligned primitives such as the AES, where truncated differentials make the inbound essentially free: the active S-boxes are matched independently with on average one solution each, and the strong alignment even lets the controlled part span two nonlinear layers. Sponge permutations of the KECCAK family, including Thunder- p , are by contrast *unaligned*. Their column-parity-type linear layer spreads every difference across the whole state, there is no truncated differential to steer the propagation, and

the inbound cannot be extended beyond a single nonlinear layer. The middle match is therefore no longer free, so the rebound attack is not a well-suited tool against these designs.

On the KECCAK- f permutations the best rebound result is due to Duc et al. [DGPW12]. They construct low-weight differential trails inside the column-parity kernel, where θ acts as the identity, and run the inbound meet-in-the-middle over a *single* χ layer of $s = 5w = 320$ parallel 5-bit S-boxes. Because the weak alignment prevents two consecutive rounds from being merged into independent SuperSboxes, the inbound is not amortized for free: a random difference pair connects through χ only with a small probability, and this factor dominates the cost. The result is a *distinguisher* on the internal permutation rather than a collision on the hash, reaching 8 rounds of KECCAK- f [1600] at a cost of about $2^{491.47}$. Moreover, the starting differences of these characteristics cannot be reached from valid sponge initial states, so the distinguisher does not translate into an attack on the KECCAK hash.

Thunder- p is a strictly harder target on both sides of this attack. On the trail side, its diffusion layer θ is a Symmetric Circulant Twin Column Parity Mixer with differential and linear branch number 12, against only 4 for the conventional column-parity mixer used in KECCAK- f . Since the differential branch number of θ is 12 and θ acts as the identity on the kernel, every nonzero kernel difference has at least 6 active columns ($2 \times 6 = 12$); the very-low-weight kernel trails—as light as two active columns—that drive the KECCAK- f search are therefore absent, and every nonzero trail already activates far more S-boxes, pushing the outbound probabilities p_B and p_F down. On the inbound side, the S-box ψ has algebraic degree 3 instead of 2, so its DDT is not row-flat: the nonzero entries vary within a row and take several distinct values (overall $\{2, 4, 6, 8, 14\}$, with differential uniformity 14). The equiprobable, freely enumerable output differences that the degree-2 χ of KECCAK- f offers are thus unavailable, which makes the inbound match still more expensive. Even on the easier KECCAK- f [1600] the rebound attack yields only an 8-round distinguisher on the permutation, not an attack on the hash; against the harder Thunder- p —with no low-weight kernel trails and a non-flat DDT that inflates the inbound cost—it reaches still fewer rounds and likewise stops short of a collision. We therefore expect the rebound attack to pose no threat to Thunder.

4.4.5 Internal Differential Attack

Internal Differential Let the internal state of the sponge construction be of width b bits, logically partitioned along the z -axis into i equal-sized slices, where i is a divisor of 64. Denote the j -th slice of a state A as $A^{(j)}$. An **internal differential**, in the sense introduced by Peyrin [Pey10], is defined by the condition that any two adjacent slices differ by a fixed XOR difference $\delta \in \{0, 1\}^{b/i}$:

$$A^{(j)} \oplus A^{(j+1)} = \delta, \quad \forall j \in \{0, 1, \dots, i-1\}, \quad (13)$$

where the slice index is computed modulo i . Traversing the i slices and returning to the first one requires the per-step difference δ to cancel over the full cycle; since the slice count i is even (it divides 64, with $i \geq 2$), δ is repeated an even number of times and cancels for every δ , so the definition closes consistently.

By restricting the input message block such that the initial internal state satisfies this condition, the adversary effectively reduces the input space to a structured subset. The internal difference δ then propagates through the round function. After r rounds, if the output difference falls into a predefined set $\mathcal{S}$ of size $2^{|\mathcal{S}|}$ with probability p , a collision can be constructed via the birthday paradox with expected data complexity $O(p^{-1}2^{|\mathcal{S}|/2})$ and comparable time complexity. This squeeze formulation is the generalized internal differential of Dinur, Dunkelman and Shamir [DDS14]. Our design thwarts this attack from two aspects.

Asymmetric Initial Vector Blocks the Internal-Differential Subspace. Thunder replaces Keccak’s all-zero capacity with a fixed, nonzero initial vector. The internal-differential condition must hold on the *whole* state, the capacity slices included; but the capacity is not message-controlled — it is fixed to the initial vector — so its inter-slice differences $A_c^{(j)} \oplus A_c^{(j+1)}$ are pinned to a constant pattern that the adversary can neither choose nor cancel. The decisive property is therefore not the Hamming weight of the initial vector but its z -asymmetry: a high-weight yet z -symmetric constant (all slices equal) has zero internal difference and would offer no protection, whereas a z -asymmetric initial vector pins the capacity inter-slice differences to a nonconstant pattern. Unless the initial vector is z -symmetric, its capacity slices cannot satisfy $A_c^{(j)} \oplus A_c^{(j+1)} = \delta$ for a single δ , so no message places the initial state in the z -symmetric subspace on which the internal differential propagates, and the entry condition of the attack fails outright. Even granting an approximate match, the fixed capacity forces a high-weight internal difference into the trail from the first round, so full diffusion activates many S-boxes and drives the differential probability p down. Finally, under the i -slice constraint $A^{(j)} = A^{(0)} \oplus j\delta$ leaves only one slice of freedom in the rate, so the available message space is about $2^{r/i}$ and shrinks as the slice count i grows; hence $p^{-1}2^{|S|/2} \gg 2^{r/i}$ and the required data far exceeds it.

Cubic S-Box Destroys the Linear Structure of the Output Difference Space. Keccak’s quadratic S-box ensures that the set of output differences for a fixed input difference forms an affine linear space, enabling symbolic extension of outputs satisfying the internal differential over multiple rounds while precisely determining $|S|$. Our design adopts an S-box of algebraic degree 3, reducing the output difference set to a nonlinear algebraic variety. Because the degree-3 derivative $x \mapsto \psi(x) \oplus \psi(x \oplus \Delta)$ is quadratic rather than affine, this set is in general not an affine subspace, which removes the structure that the symbolic, multi-round internal-differential extension of [DDS14] relies on and raises the cost of propagating the difference and of determining $|S|$. The set remains a quadratic variety rather than becoming formally intractable — conditional internal-differential variants can still address non-affine sets — but the equiprobable, freely enumerable output differences that the degree-2 χ offers, and which those attacks exploit, are no longer available.

Taken together, the asymmetric initial vector denies the attack its entry condition and the degree-3 S-box removes the affine structure it relies on, so internal-differential collision attacks pose no threat to Thunder.

4.4.6 Preimage attack based on DL distinguishers

Niu et al. [NHS⁺24] observed that highly biased differential-linear distinguishers accelerate the search for a preimage of a candidate one-way function. We apply their framework to Thunder with the distinguishers of Table 13.

Let F map the last absorbed message block to the h -bit digest, with the remaining message blocks fixed. A preimage of a target digest O is an input x with $F(x) = O$. Assume a set $D = \{\delta_0, \dots, \delta_{s-1}\}$ of input differences, and for each δ_i a set of ℓ_i linearly independent output masks $\lambda_{i,j}$ such that $(\delta_i, \lambda_{i,j})$ is a DL distinguisher of F with correlation $c_{i,j}$. For a random x with $y = F(x)$, the relation $\lambda_{i,j} \cdot (y \oplus O) = \zeta_{i,j}$ is necessary for $x \oplus \delta_i$ to be a preimage, where $\zeta_{i,j}$ is the value towards which the distinguisher is biased. The evaluation of $F(x \oplus \delta_i)$ is omitted whenever this relation fails. Under the strictest filtering strategy the exhaustive search over 2^h candidates is accelerated by the factor

$$\frac{1 + \sum_i p_i}{1 + \sum_i 2^{-\ell_i}}, \quad p_i = \prod_{j=0}^{\ell_i-1} \left(\frac{1}{2} + \frac{|c_{i,j}|}{2} \right),$$

where $\sum_i 2^{-\ell_i}$ is the expected number of differences that survive the filter for one guess.

The differences and masks are constrained by the mode. The message is absorbed into the rate, so an input difference δ_i confined to the rate leaves the fed-forward capacity X_c unchanged. Its contribution then cancels in the digest difference, and the DL correlation of F coincides with that of the bare permutation **Thunder- p** for δ_i in the rate and $\lambda_{i,j}$ in the digest. The distinguishers of Table 13 therefore transfer to F without modification.

The speed-up of this strategy is governed by the magnitude of the differential-linear correlations $c_{i,j}$, and is meaningful only when high-correlation distinguishers exist. For **Thunder- p** the best differential-linear correlation collapses rapidly with the number of rounds, and from four rounds onward no high-correlation differential-linear distinguisher remains. Against the full 12-round **Thunder** the strategy therefore offers no advantage over exhaustive preimage search, so differential-linear distinguishers pose no threat to the preimage security of **Thunder**.

4.4.7 Preimage attack based on Linear-Structure

This subsection studies whether the linear-structure technique can be transferred to **Thunder- p** . Since the full θ layer of **Thunder- p** contains two column-parity planes P and Q , the direct analysis of all parity interactions is complicated. Therefore, we first consider a weakened variant of **Thunder- p** , denoted by **Thunder- p_P** , in which the Q -dependent part is omitted and only the P -parity plane is kept. This gives a conservative setting for evaluating the applicability of Keccak-type linear structures.

Let the state be denoted by $D[x][y][z]$, where $x, y \in \mathbb{Z}_5$ and $z \in \mathbb{Z}_{64}$. In **Thunder- p_P** , the θ operation is reduced to

$$P[x][z] = \bigoplus_{y=0}^4 D[x][y][z],$$

$$E[x][z] = P[x][z] \oplus P[x-1][z+r],$$

and

$$D'[x][y][z] = D[x][y][z] \oplus E[x][z+u].$$

Since this operation is linear, it does not increase the algebraic degree of the state bits. The purpose of the linear structure is to choose a subspace of input states such that the column parities $P[x][z]$ are constants. Under this condition, the value injected by θ is also constant, and the variables are not diffused to other columns by the P -parity computation.

For a fixed column $x = x_0$, let

$$D[x_0][y][z], \quad y = 0, 1, 2, 3,$$

be free variables, and define

$$D[x_0][4][z] = \bigoplus_{y=0}^3 D[x_0][y][z] \oplus \alpha[z],$$

where $\alpha[z]$ is a fixed constant. Then

$$P[x_0][z] = \bigoplus_{y=0}^4 D[x_0][y][z] = \alpha[z],$$

which is independent of the free variables. All other lanes are fixed to constants. Therefore, for every z , the P -parity of the active column is constant. The dimension of this variable space is

$$4 \times 64 = 256.$$

It remains to check the nonlinear layer. The S-box of **Thunder- p** is defined by

$$D'[x][y][z] = D[x][y+3][z] \oplus \overline{D[x][y+1][z]} D[x][y+2][z] \overline{D[x][y+4][z]}.$$

Equivalently, if we denote the five input bits of a Thunder S-box by

$$a_y = D[x][y][z],$$

then the nonlinear part of the y -th output bit is

$$\overline{a_{y+1}} a_{y+2} \overline{a_{y+4}}.$$

Thus, the algebraic degree can increase only when at least two independent variables occur among

$$a_{y+1}, \quad a_{y+2}, \quad a_{y+4}.$$

Consequently, if the variables are arranged so that for every S-box and every output coordinate y , at most one of

$$a_{y+1}, \quad a_{y+2}, \quad a_{y+4}$$

is variable, then the product term becomes an affine function of the variables. The linear term a_{y+3} does not increase the algebraic degree. Hence the whole S-box remains affine on this restricted variable space.

Therefore, under the P -only θ layer, if the subsequent linear coordinate transformations place the above 256 variables so that each Thunder S-box contains at most one variable in the nonlinear positions, then one forward round of **Thunder- p_P** is linear over this 256-dimensional affine subspace.

The same observation also applies in the backward direction. Therefore, in the P -only setting, this linear structure covers one forward round and one backward round from the chosen middle state. Equivalently, it gives a two-round linearized core for reduced **Thunder- p_P** . For a preimage attack, the attacker can place the unknown variables in this middle-state affine subspace, propagate them linearly to both the input and output boundaries, and then translate the message-format constraints and the target digest constraints into linear equations over $\mathbb{F}_2$. Hence, for the simplified **Thunder- p_P** variant, the direct consequence of this construction is a preimage attack on two reduced rounds, where the main computation is solving a linear system rather than exhaustively searching over all possible inputs. This does not directly imply a three-round or four-round preimage attack, because such an extension would require additional linearization conditions across another nonlinear layer.

If, at the middle state, each inverse S-box receives at most one variable bit, then the inverse S-box is affine on this restricted space, because every output coordinate of the inverse S-box is a Boolean function of at most one variable. Since all remaining inverse operations are linear or affine, one inverse round is also linear on the same type of structure.

The attack starts from a carefully chosen middle state satisfying the column-parity constraints, propagates one round forward and one round backward without increasing the algebraic degree, and then uses the resulting affine expressions to construct either zero-sum distinguishers or linearized preimage equations for round-reduced variants.

For the full **Thunder- p** permutation, however, the Q -parity plane must also be considered. The full θ layer computes

$$Q[x][z] = \bigoplus_{y=0}^4 D[x][y][z + t_y],$$

and injects

$$F[x][z] = Q[x+1][z] \oplus Q[x-2][z+s].$$

Thus, the above linear structure is directly valid for the full **Thunder- p** only if the selected variables also satisfy

$$Q[x][z] = \text{constant}$$

for all involved x and z . If this additional condition is not satisfied, the Q -dependent term may propagate variables to additional S-boxes, and the one-round linearization is no longer guaranteed. Therefore, the P -only attack should be interpreted as a simplified and conservative applicability test of Keccak-type linear structures, while the full **Thunder- p** design gains additional resistance from the second shifted parity plane Q .

4.4.8 Meet-in-the-Middle Preimage Attacks

We assess the applicability of meet-in-the-middle (MITM) preimage attacks to **Thunder**, following the superposition MITM framework of Bao et al. [BGST22] developed for AES-like hashing.

Pseudopreimage setup. A MITM pseudopreimage attack targets the inner permutation P directly. Given a target digest $T \in \{0, 1\}^h$, the attacker seeks a capacity state $c' \in \{0, 1\}^c$ and a rate block $m \in \{0, 1\}^r$ such that

$$\text{left}_h(\text{right}_c(P(m||c')) \oplus c') = T.$$

The attack fixes a middle round and propagates states forward from the input and backward from the output, matching at the chosen round boundary. We write s for the base-2 logarithm of the cost of one pseudopreimage search.

Conversion to a full preimage. Let K be the number of backward pseudopreimage pairs (c', m) generated in the backward phase, and let Q be the number of capacity states produced by forward absorption from the initial vector IV . A match occurs when a forward capacity state equals some c' in the backward set; the expected match condition is $K \cdot Q \geq 2^c$. The total cost is

$$K \cdot 2^s + Q.$$

Minimising over K subject to $K \cdot Q = 2^c$ gives the optimal allocation

$$K = 2^{(c-s)/2}, \quad Q = 2^{(c+s)/2},$$

and a total attack cost of

$$2^{(c+s)/2+1}.$$

Effectiveness bound. For **Thunder**, the fixed-output instances satisfy $(h, c) \in \{(512, 576), (768, 832), (1024, 1088)\}$, giving $c = h + 64$ in every case. The brute-force preimage cost is 2^h . For the attack to improve on brute force, we require

$$2^{(c+s)/2+1} < 2^h \iff c + s < 2h - 2 \iff c - s > 130,$$

where the last equivalence uses $h = c - 64$. In other words, the pseudopreimage must cost below $2^{c-130} = 2^{h-66}$, i.e. more than 66 bits below the 2^h cost of a generic pseudopreimage, before it can be converted into a preimage attack against **Thunder**.

Structural barriers. Two properties of *Thunder- p* obstruct the construction of a long-round MITM path. The linear layer of *Thunder- p* generalises the KECCAK column-parity design and operates at the bit level: every active bit fans out across the full state within a single round, leaving no lane or column that can propagate independently through several rounds. This prevents the attacker from identifying a consistent forward-backward decomposition that reaches a nontrivial round boundary. Additionally, the *Thunder- p* S-box has algebraic degree 3, higher than the quadratic χ of KECCAK- f ; the internal state degree grows faster with depth, and the algebraic conditions required to match the forward and backward computations become overdetermined after a few rounds. This framework’s closest published target, the Grøstl output transformation $\text{trunc}(P(H) \oplus H)$, is structurally identical to our feed-forward, yet its MITM attacks rely on the byte-level neutral words of an AES-like round; the bit-level full diffusion of *Thunder- p* admits no such neutral words, so the technique does not transfer.

Taken together, constructing a MITM path that achieves $c - s > 130$ against the full *Thunder- p* is beyond the reach of current techniques, and MITM preimage attacks do not threaten Thunder.

4.4.9 Quantum Security

(Second) preimage attacks. In the quantum random-permutation setting used above, Eq. (11) gives $h/2$ -bit generic preimage security for the fixed-output hash. In the preimage and second-preimage discussion below, write H for the fixed-output hash instance under discussion. The matching generic attack is Grover search [Gro96]. For a candidate message family S and a target digest O , the marked messages are those satisfying $H(M) = O$. Grover search starts from the uniform superposition over S , applies a phase oracle for this condition, and then applies the diffusion reflection. When the marked fraction is about 2^{-h} , the Grover cost is

$$T_{\text{Grover}} \approx 2^{h/2}.$$

If a reversible preimage predicate increases the marked fraction by a factor 2^σ , after its success probability is included, the corresponding quantum cost is

$$T_{\text{Grover}}(\sigma) \approx \sqrt{2^h/2^\sigma} = 2^{(h-\sigma)/2}.$$

This conversion requires a reversible implementation of the filter and final check.

For the 12-round *Thunder* instances, the $h/2$ bound gives generic Grover costs 2^{256} , 2^{384} , and 2^{512} for *Thunder-512*, *Thunder-768*, and *Thunder-1024*, respectively. In the *Thunder-512* case, one absorbed block already supplies at least h variable bits. In the *Thunder-768* and *Thunder-1024* cases, the generic search uses enough padded rate blocks to obtain message families of size at least 2^h . The reduced-round DL conversion below is separate from these full-instance costs.

The DL preimage attack of Niu et al. [NHS⁺24] gives a reduced-round filter that carries over to the quantum setting. If a reduced-round DL filter is instantiated over a sufficiently large message family, amplitude amplification gives $2^{(h-\sigma_{\text{DL}})/2}$ evaluations of the reduced-round permutation, plus the cost of the reversible filter, where $2^{\sigma_{\text{DL}}}$ is the net gain retained by the filter. The size of σ_{DL} is governed by the available differential-linear correlations, which for *Thunder- p* collapse rapidly with the number of rounds. As in the classical analysis, no high-correlation differential-linear distinguisher remains from four rounds onward, so σ_{DL} stays negligible and the quantum cost stays at the generic $2^{h/2}$. This conversion therefore offers no advantage against the full 12-round *Thunder* and does not affect the 12-round costs above.

For second preimages, Eq. (12) is the applicable quantum bound. Given a challenge message M^* , the adversary must find $M' \neq M^*$ with the same digest. For message lengths

for which the ℓ - and $\ell_{\max}$ -dependent terms in this bound are below the Grover term, the generic estimate remains $T_{2\text{nd}} \approx 2^{h/2}$.

For reduced-round second preimages, the DL preimage conversion gives only a conditional attack-cost estimate. It also requires an additional disjointness condition. Given M^* , set $O^* = H(M^*)$. If there is a message family S^* with $M^* \notin S^*$ on which the same DL filter and final check are reversible, then any marked $M' \in S^*$ satisfying $H(M') = O^*$ is a second preimage. Under this condition, the quantum cost is $T_{\text{DL-2nd}} \approx 2^{(h-\sigma_{\text{DL}})/2}$. This is a conditional attack-cost estimate obtained from the DL preimage conversion. It assumes that this message family supports the same filter, and it remains a reduced-round estimate. For the 12-round **Thunder** instances, the attacks considered here do not lower the generic quantum preimage or second-preimage estimates. When this domination condition holds for second preimages, the expected costs for **Thunder-512**, **Thunder-768**, and **Thunder-1024** remain 2^{256} , 2^{384} , and 2^{512} , respectively.

Collision attacks. For collision search, the digest value is not fixed in advance. The relevant generic quantum attack is the BHT collision algorithm [BHT98]. It stores L hash outputs, then uses Grover search to find a fresh message whose digest hits this stored set. Balancing the table size with the Grover search cost gives

$$T_{\text{BHT}} = \Theta(2^{h/3}), \quad S_{\text{BHT}} = \Theta(2^{h/3}).$$

Later quantum collision-search analyses use the same time-memory tradeoff as the generic comparison point [CNS17]. For **Thunder-512**, **Thunder-768**, and **Thunder-1024**, the generic quantum collision exponents are 512/3, 768/3, and 1024/3, respectively.

For the dedicated comparison, we use the two-sided extension idea from classical reduced-permutation collision searches [DGPW12, MRST09] and count only the two probabilistic extensions. This gives an estimate favorable to the attacker, since internal matching, message reachability, and final-digest constraints are assigned zero cost. For a u -round forward extension and a v -round backward extension, write

$$p_{\text{F},u} = 2^{-w_{\text{F},u}}, \quad p_{\text{B},v} = 2^{-w_{\text{B},v}}.$$

Amplitude amplification then gives the optimistic collision exponent

$$K_{\text{col},u+v}^{\text{q}} = \frac{1}{2}(w_{\text{F},u} + w_{\text{B},v}) = -\frac{1}{2} \log_2(p_{\text{F},u} p_{\text{B},v}).$$

Such a quantum collision search can beat BHT only if

$$K_{\text{col},u+v}^{\text{q}} < \frac{h}{3}.$$

The ignored sponge-forward conditions can only increase the cost or make this approach inapplicable.

Table 16 evaluates these optimistic quantum collision exponents. Following the two-sided extension restrictions used by Duc et al. [DGPW12], the probabilities $p_{\text{F},u}$ and $p_{\text{B},v}$ are the effective probabilities used to filter candidate pairs after the forward and backward differential extensions are embedded into the hash collision search. We compare the combined 2 + 2-round extension and the next considered 3 + 2-round extension.

The 2 + 2-round exponents are below BHT only under the zero-cost assumptions above. For 3 + 2 rounds, the optimistic exponent exceeds BHT by 27.48, 95.43, and 171.03. This comparison already ignores internal matching, message-reachability, and final-digest constraints. Thus, within the tested two-sided extension search, the 3 + 2-round estimate is above BHT before the remaining sponge-forward conditions are counted. Consequently, this approach does not give a full 12-round quantum collision attack below the BHT costs $2^{1024/3}$, $2^{768/3}$, and $2^{512/3}$ for **Thunder-1024**, **Thunder-768**, and **Thunder-512**, respectively.

Table 16: Optimistic quantum collision exponents for Thunder two-sided extensions under zero-cost internal matching.

Instance	$p_{F,2}$	$p_{F,3}$	$p_{B,2}$	$K_{\text{col},2+2}^q$	$K_{\text{col},3+2}^q$	$K_{\text{col},3+2}^q - h/3$
Thunder-1024	$2^{-42.53}$	$2^{-705.31}$	$2^{-32.31}$	37.42	368.81	27.48
Thunder-768	$2^{-14.31}$	$2^{-688.54}$	$2^{-14.31}$	14.31	351.43	95.43
Thunder-512	$2^{-14.31}$	$2^{-669.09}$	$2^{-14.31}$	14.31	341.70	171.03

4.4.10 Resistance to Length Extension Attacks

Length extension attacks are classical attacks against Merkle–Damgård type hash functions. In such constructions, the final hash value is essentially the final chaining value. Therefore, given $H(M)$ and the length of M , an adversary may use $H(M)$ as a new chaining state and continue the compression process to compute

$$H(M \parallel \text{pad}(M) \parallel M')$$

for an attacker-chosen suffix M' , without knowing the original message M .

The Thunder hash family does not follow the Merkle–Damgård construction. Instead, it is based on the sponge-forward mode. The internal state has size

$$b = r + c = 1600,$$

where r is the rate and c is the capacity. The initial state is initialized as

$$X_0 = 0^r \parallel IV,$$

where the c -bit IV is loaded into the inner part of the state. During absorption, message blocks are injected only into the rate part, while the capacity part is not directly controlled by the adversary. For a non-final block, the update has the form

$$X \leftarrow X \oplus (M_i \parallel 0^c),$$

followed by the sponge-forward transformation

$$X \leftarrow P(X) \oplus (0^r \parallel \text{right}_c(X)).$$

For the final block, an additional constant C is injected into the inner part:

$$X \leftarrow X \oplus (M_\ell \parallel C),$$

which separates the final absorption step from the non-final absorption steps.

For the fixed-output instances Thunder-512, Thunder-768, and Thunder-1024, the output length h satisfies

$$h < c.$$

More concretely,

$$(h, c) \in \{(512, 576), (768, 832), (1024, 1088)\}.$$

Thus, the digest reveals only h bits from the inner part of the final state. It does not reveal the complete inner part, and it also does not reveal the rate part. Consequently, after seeing $H(M)$, the adversary does not know the full final state

$$X \in \mathbb{F}_2^{1600}.$$

At least

$$1600 - h$$

state bits remain unknown. In particular, the hidden part contains the whole rate part and the remaining $c - h$ bits of the inner part.

To extend the message, however, the adversary would have to continue the absorption process from the exact final state after processing M . Given only the digest $H(M)$, there are many possible internal states that are consistent with the same digest value. Without knowing which one is the correct state, the adversary cannot compute the valid digest of

$$M \parallel \text{pad}(M) \parallel M'$$

by simply continuing the computation from $H(M)$. Therefore, the digest of **Thunder** cannot be used as a chaining value in the same way as in a Merkle–Damgård hash function.

Moreover, the sponge-forward feed-forward step further prevents a direct state-continuation interpretation. The value added back to the state is the previous inner part $\text{right}_c(X)$, while only $h < c$ output bits are released at the end. Hence, even the released digest is only a partial view of the inner state after several nonlinear permutation and feed-forward updates, rather than a full chaining state that can be reused for further absorption.

Therefore, the classical length extension attack is not applicable to **Thunder**. Any attempt to extend a message from a known digest would first require recovering the correct full internal state after absorbing the original message. This reduces the problem to an internal-state recovery or preimage-type problem, rather than a standard length extension attack. For the three fixed-output instances, the amount of unrevealed state is

$$1600 - 512 = 1088, \quad 1600 - 768 = 832, \quad 1600 - 1024 = 576$$

bits for **Thunder-512**, **Thunder-768**, and **Thunder-1024**, respectively. These hidden-state sizes prevent the digest from serving as a continuable chaining value.

It should also be noted that the extendable-output functionality of the XOF instances is different from a length extension attack. In an XOF, requesting a longer output for the same input message is an intended functionality. It does not allow an adversary, given the output on M , to compute the hash or XOF output on an extended input $M \parallel M'$. Thus, the XOF property should not be confused with the classical message length extension weakness.

5 Performance

5.1 Software performance

We evaluate the software performance of **Thunder** on two representative 64-bit software platforms. The evaluation covers the three fixed-output hash instances, **Thunder-512**, **Thunder-768**, and **Thunder-1024**. The primary metric is cycles per byte (cpb), computed from the measured cycle count divided by the input length. Lower cpb values indicate higher software efficiency.

Design-level analysis. The core operations of **Thunder** mainly consist of ANDN, rotations, and XORs, making the design well suited to 64-bit general-purpose processors. To further improve software efficiency, we optimize the implementation by jointly scheduling the ρ , θ , and ψ operations according to the instruction-level characteristics of different processor architectures.

On the target x86 platform, rotate instructions can be scheduled only to a small number of rotate-capable execution ports. In particular, on our benchmark platform, only two execution ports support such operations. Therefore, a sequence containing many

consecutive **ROR**/**ROL** instructions may suffer from port contention, which limits instruction-level parallelism. Since the ρ operation of **Thunder** contains a large number of rotations, and the computation of the parity planes P and Q in the θ operation mainly consists of XORs and rotations, we interleave these computations instead of executing all rotations in a concentrated manner. In this way, rotate instructions and logical instructions can be issued to different execution resources more effectively. Moreover, since the intermediate values E and F depend only on P and Q , respectively, there is no data dependency between their computations, and they can therefore be computed in parallel to further improve pipeline utilization.

On the ARM platform, we exploit the barrel-shifter feature to optimize the computation of Q . In the original definition, Q is computed by rotating different state words according to the offsets t_y and then XORing the rotated values. To reduce the number of explicit **ROR** instructions, we recompute the rotation offsets and rewrite the computation of Q into the equivalent nested form

$$Q' = s_4 \oplus \text{ROR}(s_3 \oplus \text{ROR}(s_2 \oplus \text{ROR}(s_1 \oplus \text{ROR}(s_0, i_0), i_1), i_2), i_3).$$

This form makes better use of ARM instructions with shifted operands, allowing part of the rotation to be combined with the XOR operation in a single instruction. Meanwhile, the output of the θ operation uses the term $F[x][z + u + t_y]$. Therefore, the final rotation in the computation of Q can be merged with the t_y rotation required in the output stage of θ , thereby eliminating one explicit **ROR** instruction. This optimization does not change the mathematical definition of θ ; it only merges and reschedules equivalent rotations at the implementation level.

In addition, we interleave the output computation of θ with the subsequent ψ operation. Once part of the θ output state has been generated, it can immediately be consumed by the nonlinear ψ operation, avoiding unnecessary waiting time and reducing the need to materialize and store the entire intermediate state. Finally, two adjacent rounds are combined into a single routine at the implementation level, which reduces parameter-passing overhead, function-call overhead, and intermediate-state storage. This also helps keep more temporary variables in registers.

At the cycle-count level, the computation of P and Q takes about 45 cycles per round, the computation of E and F takes about 15 cycles, and the ψ operation takes about 15 cycles. Therefore, the design-level estimate gives approximately 75 cycles per round. The measured value is about 74 cycles per round, which is consistent with the theoretical port-throughput analysis. Compared with the x86 platform, the ARM implementation is slightly slower because higher register pressure leads to stack spilling and additional memory accesses.

Benchmark setup. The ngcc benchmark was run in single-core mode on the two platforms summarized in Table 17. The test set uses the S1–S8 message sizes specified by the self-assessment guidelines, namely 32, 128, 512, 1024, 4096, 8192, 16384, and 65536 bytes. The measured cpb values are reported in Tables 18 and 19.

Table 17: Software benchmark platforms.

Item	x86-64 platform	ARM platform
Processor	Intel Xeon(R) Platinum 8358 CPU @ 2.60 GHz	Rockchip RK3588
Operating system	Ubuntu 22.04 LTS (kernel 6.8.0)	Ubuntu 22.04 (kernel 5.10.160)
Compiler	GCC 11.4	GCC 9.4
Build tool	CMake 3.22.1	CMake 3.16.3
Benchmark mode	Single core	Single core

Table 18: Software performance comparison of SHA3-512, SM3(256), and the x86-64 implementation of Thunder on Intel Xeon(R) Platinum 8358. Values are cycles per byte (cpb); lower is better.

Instance	S1 (32 B)	S2 (128 B)	S3 (512 B)	S4 (1024 B)	S5 (4096 B)	S6 (8192 B)	S7 (16384 B)	S8 (65536 B)
SM3(256)	70.06	36.74	14.43	9.28	8.59	8.51	8.47	8.45
SHA3-512	103.73	34.97	20.43	13.46	12.49	12.48	12.47	12.46
Thunder-512	34.92	17.90	10.11	8.52	7.32	7.12	7.02	6.95
Thunder-768	34.91	18.03	12.04	10.44	9.60	9.47	9.36	9.28
Thunder-1024	36.04	25.73	17.13	15.49	14.22	14.00	13.90	13.81

Table 19: Software performance comparison of SHA3-512, SM3(256), and the ARM implementation of Thunder on Rockchip RK3588. Values are cycles per byte (cpb); lower is better.

Instance	S1 (32 B)	S2 (128 B)	S3 (512 B)	S4 (1024 B)	S5 (4096 B)	S6 (8192 B)	S7 (16384 B)	S8 (65536 B)
SM3(256) [†]	94.80	20.44	11.00	9.42	8.26	8.05	7.95	7.88
SHA3-512 [†]	67.58	25.59	19.41	17.32	15.74	15.60	15.54	15.49
Thunder-512	62.96	23.36	11.48	9.52	8.05	7.79	7.68	7.62
Thunder-768	63.58	23.45	13.44	11.44	10.42	10.28	10.17	10.14
Thunder-1024	63.82	31.17	19.17	17.18	15.68	15.42	15.30	15.27

[†] The SHA3-512 baseline is a portable software implementation and does not use the Armv8.2 SHA-3 instructions (EOR3, RAX1, XAR, BCAX); x86-64 provides no SHA-3 instruction extension.

[‡] The SM3 baseline is a portable software implementation and does not use the Armv8.2 SM3 instructions (SM3SS1, SM3TT1A/1B, SM3TT2A/2B, SM3PARTW1/2); x86-64 provides no SM3 instruction extension.

Observations. The results show a consistent decrease in cpb as the input length increases. This is the normal profile of a hash implementation with non-negligible fixed costs for initialization, padding, finalization, and benchmark-loop overhead: these costs dominate short inputs, but are gradually amortized for longer messages. Consequently, the S1 results are relatively close across the three Thunder instances on each platform, whereas the long-message results more clearly reflect the steady-state processing cost of each parameter set.

Compared with SHA3-512, Thunder-512 and Thunder-768 achieve lower cpb at every tested input length on both platforms. On the x86-64 platform, the S8 cpb values are 6.95 for Thunder-512 and 9.28 for Thunder-768, both below the 12.46 cpb measured for SHA3-512. On the ARM platform, the corresponding S8 values are 7.62 and 10.14 cpb, again below the 15.49 cpb measured for SHA3-512. Thus, Thunder-512 provides the best raw speed among the three Thunder instances, while Thunder-768 remains clearly faster than the SHA3-512 baseline despite providing a larger digest size.

Compared with SM3(256), the results show that Thunder-512 is also competitive, especially on x86-64 and for long messages. On the x86-64 platform, Thunder-512 is faster than

SM3(256) for all tested input lengths. For example, at S8, Thunder-512 reaches 6.95 cpb, whereas SM3(256) reaches 8.45 cpb. On the ARM platform, Thunder-512 is faster than SM3(256) for the shortest input S1 and for long inputs from S5 to S8. At S8, Thunder-512 reaches 7.62 cpb, slightly below the 7.88 cpb of SM3(256). For short and medium inputs S2–S4 on ARM, SM3(256) is faster, mainly because the fixed per-call cost of Thunder is more visible before being amortized over longer messages.

For Thunder-768 and Thunder-1024, the comparison with SM3(256) is naturally less favorable because these two instances target larger output lengths and larger security margins than SM3(256). On x86-64, Thunder-768 is faster than SM3(256) for S1–S3, but becomes slower from S4 onward. At S8, Thunder-768 requires 9.28 cpb, compared with 8.45 cpb for SM3(256). Thunder-1024 is faster than SM3(256) only for the shortest input S1 on x86-64, and is slower for the remaining tested lengths. On ARM, both Thunder-768 and Thunder-1024 are generally slower than SM3(256) except at the shortest input length. This is consistent with their smaller effective rates and larger digest sizes.

For Thunder-1024, the comparison with SHA3-512 is more platform- and message-size-dependent. On x86-64, Thunder-1024 is faster than SHA3-512 for short and medium inputs up to S3, but becomes slower from S4 onward because the larger-output instance has a smaller effective rate and therefore requires more permutation calls for long messages. At S8, Thunder-1024 reaches 13.81 cpb, which is higher than the 12.46 cpb of SHA3-512. On ARM, however, Thunder-1024 remains close to SHA3-512 and is faster for all tested lengths except S2; at S8, it reaches 15.27 cpb, slightly below the 15.49 cpb of SHA3-512.

The measurements also show a clear and predictable scaling pattern within the Thunder family. For Thunder-512, the x86-64 cpb decreases from 34.92 at S1 to 6.95 at S8, while the ARM cpb decreases from 62.96 to 7.62. For Thunder-768, the x86-64 cpb decreases from 34.91 to 9.28, and the ARM cpb decreases from 63.58 to 10.14. For Thunder-1024, the corresponding values decrease from 36.04 to 13.81 on x86-64 and from 63.82 to 15.27 on ARM. This indicates that the main performance penalty for short inputs comes from fixed per-call overhead, whereas the long-message performance is mainly determined by the rate and the cost of the underlying permutation.

Summary. Overall, the measurements indicate that Thunder is efficient in software on both x86-64 and ARM 64-bit platforms. Its per-byte cost drops quickly once the fixed overhead is amortized, and the long-message cpb remains moderate for all three fixed-output instances. The direct SHA3-512 comparison highlights the software efficiency of the two lower-cost instances: Thunder-512 and Thunder-768 outperform SHA3-512 across the full S1–S8 range on both evaluated platforms. The largest instance, Thunder-1024, incurs the expected additional software cost due to its smaller effective rate. On x86-64, it is faster than SHA3-512 for S1–S3, but becomes slower from S4 onward. On ARM, it remains close to SHA3-512 and is faster for all tested lengths except S2. These results show that the smaller and medium Thunder instances provide clear software advantages over SHA3-512, while Thunder-1024 remains competitive despite its higher security level and reduced rate.

Compared with SM3(256), Thunder-512 shows the strongest performance among the Thunder instances. It outperforms SM3(256) for all tested lengths on x86-64 and also becomes faster than SM3(256) for long messages on ARM. The larger Thunder-768 and Thunder-1024 instances are not intended to minimize cpb against a 256-bit-output hash baseline; instead, they provide larger digest sizes and larger security margins, and therefore incur additional software overhead. The results exhibit a clear performance hierarchy: Thunder-512 gives the best raw speed, Thunder-768 provides an intermediate trade-off, and Thunder-1024 achieves the largest output length and security margin at the cost of higher software cost.

5.2 Hardware performance

To provide a comprehensive evaluation of the hash function in hardware, we implement its 512-, 768-, and 1024-bit versions in an ASIC setting. Table 20 reports the area, critical-path delay, frequency, and power, together with the cycle count, throughput, and throughput-to-area ratio for message lengths of 32, 1024, and 65536 bytes.

According to the algorithm specification, we implement the complete 512-, 768-, and 1024-bit hash functions. Their underlying permutations adopt a round-based architecture that executes one round per cycle and iteratively reuses the round logic. The designs are described in VHDL, functionally verified using ModelSim, and synthesized using Synopsis Design Compiler with a commercial UMC 55nm standard-cell library. During synthesis, the timing constraint is progressively adjusted to maximize throughput. Table 20 reports the resulting area, critical-path delay, frequency, and power, together with the total cycle count, throughput, and throughput-to-area ratio for message lengths of 32, 1024, and 65536 bytes, representing short, medium-sized, and long messages, respectively.

As shown in Table 20, both the throughput and throughput-to-area ratio of Thunder increase with the message length. In particular, Thunder-512 achieves an overall performance advantage over SHA3-512, especially for medium-sized and long messages. More specifically, since the underlying permutation of Thunder requires fewer rounds than that of SHA3-512, Thunder-512 reduces the cycle count by 32%, 49%, and 50% for message lengths of 32, 1024, and 65536 bytes, respectively. Despite its slightly longer critical-path delay of 0.93 ns compared with 0.84 ns for SHA3-512, these cycle reductions lead to throughput improvements of 32%, 77%, and 82% for short, medium-sized, and long messages, respectively. For the throughput-to-area ratio, Thunder-512 is temporarily lower than SHA3-512 for the 32-byte message because of its larger hardware area. As the message length increases and the fixed processing overhead is progressively amortized, Thunder-512 achieves throughput-to-area ratio improvements of 13% and 16% for the 1024- and 65536-byte messages, respectively. Although Thunder-768 and Thunder-1024 produce longer hash outputs, both achieve higher throughput than SHA3-512 across all three message lengths. For the long messages of 65536 bytes, they improve throughput by 51% and 33%, respectively.

Table 20: Hardware performance of SHA3-512, Thunder-512, Thunder-768, and Thunder-1024 for message lengths of 32, 1024, and 65536 bytes. These implementations are synthesized at high frequency to achieve maximum throughput.

Algorithm	Implementation				32 B			1024 B			65536 B		
	Area (kGE)	Delay (ns)	Freq. (MHz)	Power (mW)	Cycles	TP (Mbps)	TP/Area (Mbps/kGE)	Cycles	TP (Mbps)	TP/Area (Mbps/kGE)	Cycles	TP (Mbps)	TP/Area (Mbps/kGE)
SHA3-512	44.68	0.84	1190.48	31.25	38	8020	179.48	512	19048	426.27	30976	20150	450.93
Thunder-512	70.10	0.93	1075.27	47.76	26	10587	151.02	261	33749	481.42	15381	36652	522.83
Thunder-768	72.76	0.97	1030.93	50.68	30	8797	120.91	294	28726	394.82	17766	30423	418.15
Thunder-1024	84.01	0.87	1149.43	64.02	34	8654	103.02	381	24714	294.19	22557	26716	318.02

6 Discussion and Feature Statement

Thunder is built around a single idea: defense in depth. Its security rests on two layers that fail independently. The mode **Sponge- $F^{P, pd10^*}$** controls what an attacker can reach through the construction, and the permutation **Thunder- p** controls what an attacker can do inside a single call. We made each layer stronger than the security target demands, so that a weakness in one does not by itself break the family. The following statements summarize the features that distinguish Thunder and the limits of what we claim.

The mode removes the capacity bottleneck. A standard sponge ties (second) preimage resistance to half its capacity, so a 1024-bit target forces a permutation wider than 2048

bits. The feed-forward of $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ breaks this link. Feeding the inner part forward at every call turns each permutation step into a non-invertible Davies–Meyer map, which lifts (second) preimage resistance close to the output length while the capacity stays at $c \approx h + 64$. A 1600-bit permutation therefore serves every member up to Thunder-1024, where a standard sponge would need more than 2048 bits of width. The same construction yields an XOF, with preimage and collision security capped at the target level by the capacity.

Security reduces to properties of the permutation. The collision and (second) preimage security of $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ reduces, in the random-permutation model, to simple and explicit properties of the underlying permutation rather than to a monolithic indistinguishability argument. This makes the security claim auditable: the burden falls on well-defined statements about Thunder- p that can be studied directly. The mode further carries meaningful quantum collision and (second) preimage bounds [GHJ⁺25], which standard sponge hashing does not provide, and the constant injected in the final block removes the length-extension exposure that squeezing the inner part would otherwise create [AMP12, HPY12].

The permutation keeps a deep margin. Thunder- p runs 12 rounds, against which the structural attacks we examined reach far fewer. Full diffusion is reached after 3 rounds. The best differential and linear trails lose probability steeply with each added round. In the kernel space the best two-round bounds are $2^{-14.31}$ (differential) and $2^{-9.96}$ (linear squared correlation), dropping to $2^{-144.06}$ and $2^{-93.50}$ at three rounds and $2^{-781.83}$ and $2^{-637.87}$ at four. Diffusion-based impossible-differential and zero-correlation distinguishers are limited to about 6 rounds. The round count is thus close to twice the reach of the best distinguishers we found, which is the margin the name of the design is meant to convey.

A familiar geometry, a stronger linear layer. Thunder- p keeps the $5 \times 5 \times 64$ lane geometry of KECCAK- f [1600], so existing tooling, bit-slicing techniques, and modelling experience transfer directly. The mixing layer departs from it. Instead of the parity mixer of KECCAK, whose differential and linear branch numbers are 4, Thunder- p uses a symmetric circulant twin column-parity mixer [LRL⁺24] of branch number 12. The nonlinear layer ψ acts along the y -axis, orthogonal to the x -axis direction of KECCAK’s χ , so that diffusion and nonlinearity interact across different geometric directions. These two changes are what let Thunder- p reach a comparable margin in half the rounds of KECCAK- f [1600]. This halved round count is the direct source of Thunder’s throughput advantage over SHA3-512.

Implementation profile. The round function uses only word-level logic, rotations, and XORs on 64-bit lanes. It needs no dedicated cryptographic instructions such as AES-NI or SHA-NI, and the bit-sliced S-box maps the same way onto software and a round-based hardware datapath. In hardware this yields a throughput advantage over SHA3-512 at every tested message length, at the cost of a larger area. In software the cost grows with the output length, as larger-output members use smaller rates and call the permutation more often; Thunder-512 is the fastest member, and Thunder-1024 trades software speed for its higher output and security level. Thunder is designed to meet the higher security levels sought by the call: where SM3 provides a 256-bit digest, the family reaches up to 1024-bit (second) preimage and 512-bit collision resistance. In software Thunder-512 stays competitive with SM3 on the same platforms, while the higher-security members accept a moderate cost for their larger output and margin.

Scope of the security claims. The mode bounds hold in the random-permutation model. The security of Thunder- p is argued by trail bounds, diffusion, algebraic degree, and the dedicated attacks reported in the security analysis, as is standard for an unkeyed

permutation; we make no claim beyond the notions analyzed there. The fixed-output members target the preimage and collision levels listed in Table 1; the XOF members reach these levels only up to the cap imposed by the capacity. All members assume inputs of at most 2^{64} bits.

References

- [AHMN10] Jean-Philippe Aumasson, Luca Henzen, Willi Meier, and María Naya-Plasencia. Quark: A lightweight hash. In Stefan Mangard and François-Xavier Standaert, editors, *CHES 2010*, volume 6225 of *LNCS*, pages 1–15. Springer, Berlin, Heidelberg, August 2010.
- [AHY23] Prabhanjan Ananth, Zihan Hu, and Henry Yuen. On the (im)plausibility of public-key quantum money from collision-resistant hash functions. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 39–72. Springer, Singapore, December 2023.
- [AMP11] Elena Andreeva, Bart Mennink, and Bart Preneel. Security reductions of the second round SHA-3 candidates. In Mike Burmester, Gene Tsudik, Spyros S. Magliveras, and Ivana Ilic, editors, *ISC 2010*, volume 6531 of *LNCS*, pages 39–53. Springer, Berlin, Heidelberg, October 2011.
- [AMP12] Elena Andreeva, Bart Mennink, and Bart Preneel. The parazoa family: generalizing the sponge hash functions. *Int. J. Inf. Sec.*, 11(3):149–165, 2012.
- [BCC11] Christina Boura, Anne Canteaut, and Christophe De Cannière. Higher-order differential properties of Keccak and *Luffa*. In *FSE 2011*, *LNCS*, pages 252–269. Springer, 2011.
- [BDPV08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the Sponge construction. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 181–197. Springer, Berlin, Heidelberg, April 2008.
- [BDPVA07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge Functions. In *ECRYPT hash workshop*, volume 2007, 2007.
- [BGST22] Zhenzhen Bao, Jian Guo, Danping Shi, and Yi Tu. Superposition meet-in-the-middle attacks: Updates on fundamental security of AES-like hashing. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 64–93. Springer, Cham, August 2022.
- [BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In Claudio L. Lucchesi and Arnaldo V. Moura, editors, *LATIN 1998*, volume 1380 of *LNCS*, pages 163–169. Springer, Berlin, Heidelberg, April 1998.
- [BKL⁺11] Andrey Bogdanov, Miroslav Knežević, Gregor Leander, Deniz Toz, Kerem Varici, and Ingrid Verbauwhede. Spongint: A lightweight hash function. In Bart Preneel and Tsuyoshi Takagi, editors, *CHES 2011*, volume 6917 of *LNCS*, pages 312–325. Springer, Berlin, Heidelberg, September / October 2011.
- [BLP⁺08] Andrey Bogdanov, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, and Yannick Seurin. Hash functions and RFID tags: Mind the gap. In Elisabeth Oswald and Pankaj Rohatgi, editors, *CHES 2008*, volume 5154 of *LNCS*, pages 283–299. Springer, Berlin, Heidelberg, August 2008.
- [CAD⁺20] David A Cooper, Daniel C Apon, Quynh H Dang, Michael S Davidson, Morris J Dworkin, Carl A Miller, et al. Recommendation for Stateful Hash-Based Signature Schemes. *NIST Special Publication*, 800(208):800–208, 2020.

- [CDMP05] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-Damgård revisited: How to construct a hash function. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 430–448. Springer, Berlin, Heidelberg, August 2005.
- [CNS17] André Chailloux, María Naya-Plasencia, and André Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part II*, volume 10625 of *LNCS*, pages 211–240. Springer, Cham, December 2017.
- [CSS⁺18] Geoffrey Chu, Peter J Stuckey, Andreas Schutt, Thorsten Ehlers, Graeme Gange, and Kathryn Francis. Chuffed, a lazy clause generation solver. *URL: <https://github.com/chuffed/chuffed>*, 2018.
- [D⁺15] Morris J Dworkin et al. SHA-3 standard: Permutation-based hash and extendable-output functions. 2015.
- [DDS14] Itai Dinur, Orr Dunkelman, and Adi Shamir. Collision attacks on up to 5 rounds of SHA-3 using generalized internal differentials. In Shiho Moriai, editor, *FSE 2013*, volume 8424 of *LNCS*, pages 219–240. Springer, Berlin, Heidelberg, March 2014.
- [DGHM13] Gregory Demay, Peter Gaži, Martin Hirt, and Ueli Maurer. Resource-restricted indistinguishability. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 664–683. Springer, Berlin, Heidelberg, May 2013.
- [DGPW12] Alexandre Duc, Jian Guo, Thomas Peyrin, and Lei Wei. Unaligned rebound attack: Application to Keccak. In Anne Canteaut, editor, *FSE 2012*, volume 7549 of *LNCS*, pages 402–421. Springer, Berlin, Heidelberg, March 2012.
- [DKR97] Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. The block cipher Square. In Eli Biham, editor, *FSE’97*, volume 1267 of *LNCS*, pages 149–165. Springer, Berlin, Heidelberg, January 1997.
- [Foe23] Robin Foekens. Security of the Sponge Construction with a Random Transformation. Bachelor’s thesis, Radboud University, Nijmegen, 2023. https://www.cs.ru.nl/bachelors-theses/2023/Robin_Foekens___4666615___Security_of_the_Sponge_Construction_with_a_Random_Transformation.pdf.
- [GHJ⁺25] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Permutation-Based Hash from Non-Idealized Assumptions: Adding Feed-Forward to Sponge. Cryptology ePrint Archive, Paper 2025/1006, 2025.
- [GJMG11] Bertoni Guido, Daemen Joan, P Michaël, and VA Gilles. Cryptographic Sponge Functions. *2011-STMicronelectronics NXP Semiconductors, Version 0.1 January 14*, 2011.
- [GPP11] Jian Guo, Thomas Peyrin, and Axel Poschmann. The PHOTON family of lightweight hash functions. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 222–239. Springer, Berlin, Heidelberg, August 2011.

- [Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *28th ACM STOC*, pages 212–219. ACM Press, May 1996.
- [HNW26] Kai Hu, Zhongfeng Niu, and Meiqin Wang. Round-Based Approximation of (Higher-Order) Differential-Linear Correlation – A Geometric Approach Perspective. In *Advances in Cryptology - EUROCRYPTO 2026, to appear*, volume 16004 of *Lecture Notes in Computer Science*, 2026.
- [HPY12] Shoichi Hirose, Je Hong Park, and Aaram Yun. A simple variant of the Merkle-Damgård scheme with a permutation. *Journal of Cryptology*, 25(2):271–309, April 2012.
- [Jou04] Antoine Joux. Multicollisions in iterated hash functions. Application to cascaded constructions. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 306–316. Springer, Berlin, Heidelberg, August 2004.
- [KK06] John Kelsey and Tadayoshi Kohno. Herding hash functions and the Nostradamus attack. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 183–200. Springer, Berlin, Heidelberg, May / June 2006.
- [Knu94] Lars R. Knudsen. Truncated and higher order differentials. In *Fast Software Encryption: Second International Workshop. Leuven, Belgium, 14-16 December 1994, Proceedings*, LNCS, pages 196–211. Springer, 1994.
- [KS05] John Kelsey and Bruce Schneier. Second preimages on n-bit hash functions for much less than 2^n work. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 474–490. Springer, Berlin, Heidelberg, May 2005.
- [KW02] Lars R. Knudsen and David Wagner. Integral cryptanalysis. In Joan Daemen and Vincent Rijmen, editors, *FSE 2002*, volume 2365 of *LNCS*, pages 112–127. Springer, Berlin, Heidelberg, February 2002.
- [Lai94] Xuejia Lai. Higher order derivatives and differential cryptanalysis. In *Communications and Cryptography: Two Sides of One Tapestry*, pages 227–233. Springer, 1994.
- [Lef23] Charlotte Lefevre. Indifferentiability of the Sponge construction with a restricted number of message blocks. *IACR Trans. Symm. Cryptol.*, 2023(1):224–243, 2023.
- [LM95] Frank T Leighton and Silvio Micali. Large provably fast and secure digital signature schemes based on secure hash functions, July 11 1995. US Patent 5,432,852.
- [LM22] Charlotte Lefevre and Bart Mennink. Tight preimage resistance of the Sponge construction. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 185–204. Springer, Cham, August 2022.
- [LRL⁺24] Hao Lei, Raghvendra Rohit, Guoxiao Liu, Jiahui He, Mohamed Rachidi, Keting Jia, Kai Hu, and Meiqin Wang. Symmetric twin column parity mixers and their applications. *IACR Transactions on Symmetric Cryptology*, 2024(4):1–37, 2024.
- [Mer79] Ralph Charles Merkle. *Secrecy, authentication, and public key systems*. Stanford university, 1979.

- [MRST09] Florian Mendel, Christian Rechberger, Martin Schl  ffer, and S  ren S. Thomsen. The rebound attack: Cryptanalysis of reduced Whirlpool and Gr  stl. In Orr Dunkelman, editor, *FSE 2009*, volume 5665 of *LNCS*, pages 260–276. Springer, Berlin, Heidelberg, February 2009.
- [NHS⁺24] Zhongfeng Niu, Kai Hu, Siwei Sun, Zhiyu Zhang, and Meiqin Wang. Speeding up preimage and key-recovery attacks with highly biased differential-linear approximations. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IV*, volume 14923 of *LNCS*, pages 73–104. Springer, Cham, August 2024.
- [NIC25] NICCS. Announcement on launching the next-generation commercial cryptographic algorithms program, 2025. https://www.niccs.org.cn/niccs/Notice/pc/content/content_1937427069632253952.html.
- [NSB⁺07] Nicholas Nethercote, Peter J Stuckey, Ralph Becket, Sebastian Brand, Gregory J Duck, and Guido Tack. Minizinc: Towards a standard cp modelling language. In *International conference on principles and practice of constraint programming*, pages 529–543. Springer, 2007.
- [Pey10] Thomas Peyrin. Improved differential attacks for ECHO and Gr  stl. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 370–392. Springer, Berlin, Heidelberg, August 2010.
- [Pub12] Fips Pub. Secure Hash Standard (SHS). *Fips pub*, 180(4):2012, 2012.
- [RS04] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In Bimal K. Roy and Willi Meier, editors, *FSE 2004*, volume 3017 of *LNCS*, pages 371–388. Springer, Berlin, Heidelberg, February 2004.
- [RSS11] Thomas Ristenpart, Hovav Shacham, and Thomas Shrimpton. Careful with composition: Limitations of the indifferentiability framework. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 487–506. Springer, Berlin, Heidelberg, May 2011.
- [SLZ⁺25] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance. Cryptology ePrint Archive, Report 2025/963, 2025.
- [Sta10] State Cryptography Administration of China. SM3 cryptographic hash algorithm. <https://www.oscca.gov.cn/sca/xxgk/2010-12/17/1002389/files/302a3ada057c4a73830536d03e683110.pdf>, December 2010.
- [STMC⁺24] Meltem S  nmez Turan, Kerry McKay, Donghoon Chang, Jinkeon Kang, and John Kelsey. Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions. Technical report, National Institute of Standards and Technology, 2024.
- [Tod15] Yosuke Todo. Structural evaluation by generalized integral property. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 287–314. Springer, 2015.

- [Yas08] Kan Yasuda. How to fill up Merkle-Damgård hash functions. In Josef Pieprzyk, editor, *ASIACRYPT 2008*, volume 5350 of *LNCS*, pages 272–289. Springer, Berlin, Heidelberg, December 2008.

A Differential and Linear Trails of Thunder- p

Table 21: An optimal 2-round differential trail of Thunder- p with probability $2^{-14.31}$

Round	Stage	Active bit positions (x, y, z)	Active S-boxes	Weight
0	Before ψ	$\{(2, 0, 41), (2, 0, 55), (2, 2, 26), (2, 2, 61), (2, 3, 4), (2, 3, 25)\}$	6	$6 \log_2(32/14)$
0	After ψ	$\{(2, 0, 4), (2, 0, 25), (2, 2, 41), (2, 2, 55), (2, 4, 26), (2, 4, 61)\}$	6	–
1	Before ψ	$\{(0, 1, 12), (0, 1, 55), (2, 2, 41), (2, 2, 55), (4, 3, 12), (4, 3, 41)\}$	6	$6 \log_2(32/14)$
1	After ψ	$\{(0, 3, 12), (0, 3, 55), (2, 4, 41), (2, 4, 55), (4, 0, 12), (4, 0, 41)\}$	6	–
Total			12	14.31

Table 22: An optimal 2-round linear trail of Thunder- p with squared correlation $2^{-9.96}$

Round	Stage	Active bit positions (x, y, z)	Active S-boxes	Weight
0	Before ψ	$\{(2, 3, 46), (2, 3, 63), (3, 1, 42), (3, 1, 63), (4, 4, 42), (4, 4, 46)\}$	6	$12 \log_2(32/24)$
0	After ψ	$\{(4, 2, 46), (4, 2, 63), (4, 3, 42), (4, 3, 63), (4, 4, 42), (4, 4, 46)\}$	6	–
1	Before ψ	$\{(4, 2, 19), (4, 2, 36), (4, 3, 16), (4, 3, 37), (4, 4, 21), (4, 4, 25)\}$	6	$12 \log_2(32/24)$
1	After ψ	$\{(1, 4, 16), (1, 4, 37), (3, 4, 19), (3, 4, 36), (4, 4, 21), (4, 4, 25)\}$	6	–
Total			12	9.96